Human Visual System (HVS)

- Fundamental Properties of Human Vision
- From Human Vision to Perceptual Coding

- **Human Vision System (HVS)**
  - **Part I — Fundamental Properties of Human Vision**
    - Nonlinear Brightness Perception
    - Luminance vs. Chroma Sensitivities
    - Spatial Frequency Sensitivity
    - Visual Masking
  - **Part II — From Human Vision to Perceptual Coding**
    - Gamma Encoding — Exploiting Perceptual Nonlinearity
    - Chroma Sub-sampling — Exploiting Luminance vs. Chroma Sensitivities
    - How Compression Exploits the HVS

Nonlinear Brightness Perception

- **Why perception matters in imaging systems?**
  - Physical intensity ≠ perceived brightness
  - Human vision responds nonlinearly to luminance.
  - The visibility of a luminance change depends on the background intensity.
  - The same absolute change can be noticeable in a dark region but much less noticeable in a bright region.
  - **Example**
    - Case 1. Dark region: $I=10$, $\Delta I=1$ → easier to notice
    - Case 2. Bright region: $I=100$, $\Delta I=1$ → → harder to notice
  - Perception depends strongly on relative change, not only on absolute intensity difference. (Relative change in case 1 is 1/10, but in case 2 is 1/100.)

    → Same physical change ($\Delta I=1$) ≠ Same perceived change

Nonlinear Brightness Perception

- **Weber's Law:** provides a simple approximation of smallest detectable change:

$$\Delta I_{\mathrm{JND}}/I \approx k$$

  - *I:* background intensity,
  - ΔI_{JND}: smallest detectable intensity change, (**JND**: Just Noticeable Difference)
  - *k*: constant

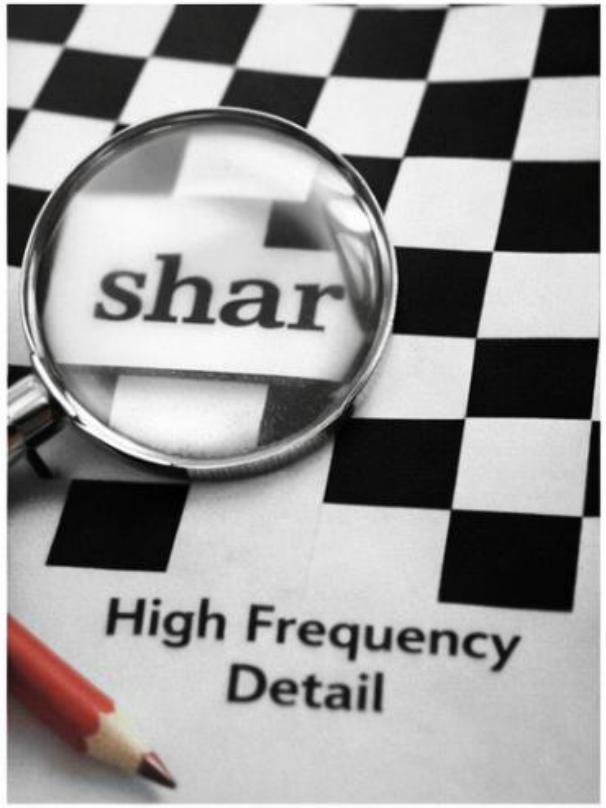
- Therefore: $\Delta I_{\mathrm{JND}} = I \times k$
- At lower background intensity, a small absolute change may already be visible.
- At higher background intensity, a larger absolute change is required to produce a similar perceptual difference.
  - Assume k=0.02 : case 1. *I = 50,* → ΔI_{JND} = 1

    case 2. *I = 200,* → ΔI_{JND} = 4
- Compression Implication → The same reconstruction error does not have the same perceptual importance at every luminance level.

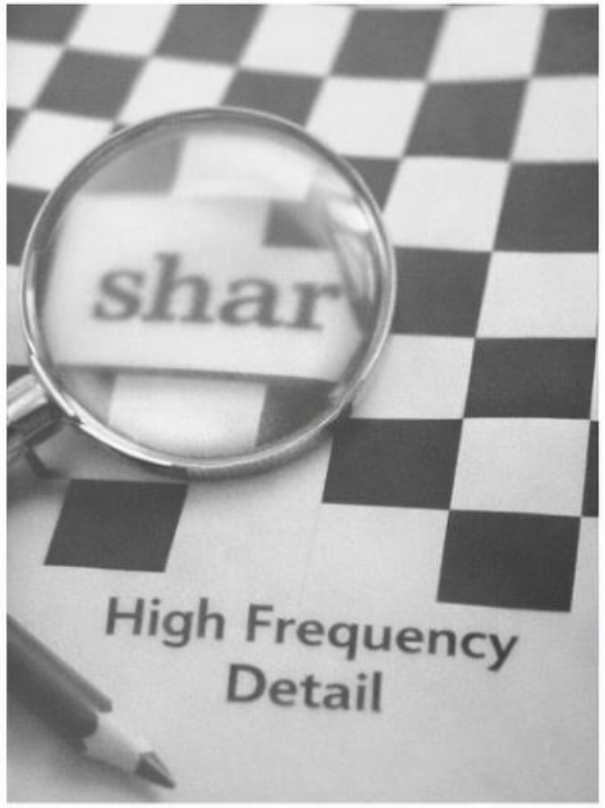
Luminance vs. Chroma Sensitivity

- **Visual Resolution**
  - Human visual system has significantly higher spatial sensitivity to luminance variations than to chrominance variations.
  - Fine spatial details, edges, and textures are therefore perceived primarily through the luminance component.
  - Reducing chroma information often has relatively little perceptual impact.
- **Biological Basis**
  - **Cone Cells**: Responsible for color and fine detail. Concentrated in the Fovea (center of the retina), but fewer in total (~6 million).
  - **Rod Cells**: Responsible for light/dark and motion. Spread across the peripheral vision and much more numerous (~120 million).
- **Compression Opportunity**
  - Human visual system is less sensitive to chroma variations, allowing chroma information to be spatially subsampled with limited perceptual impact.

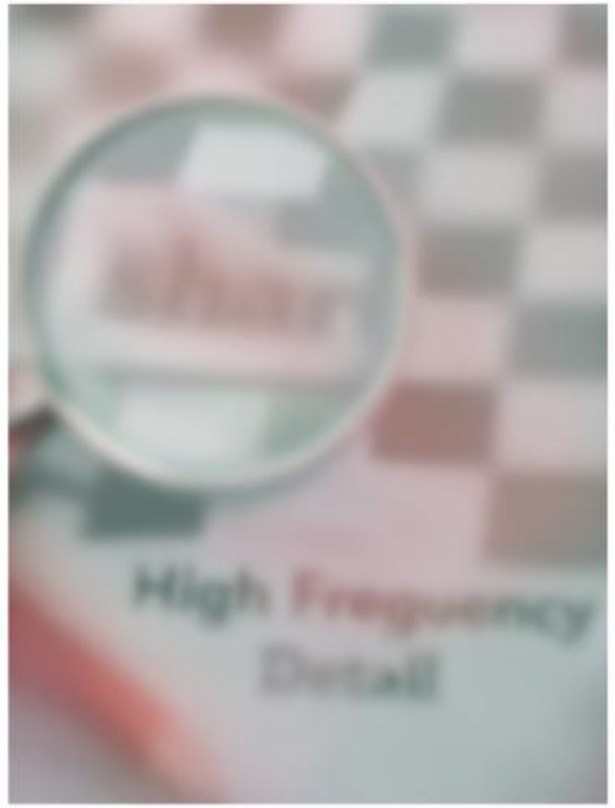
Luminance vs. Chroma



Original Image



Y-Only (Luminance)



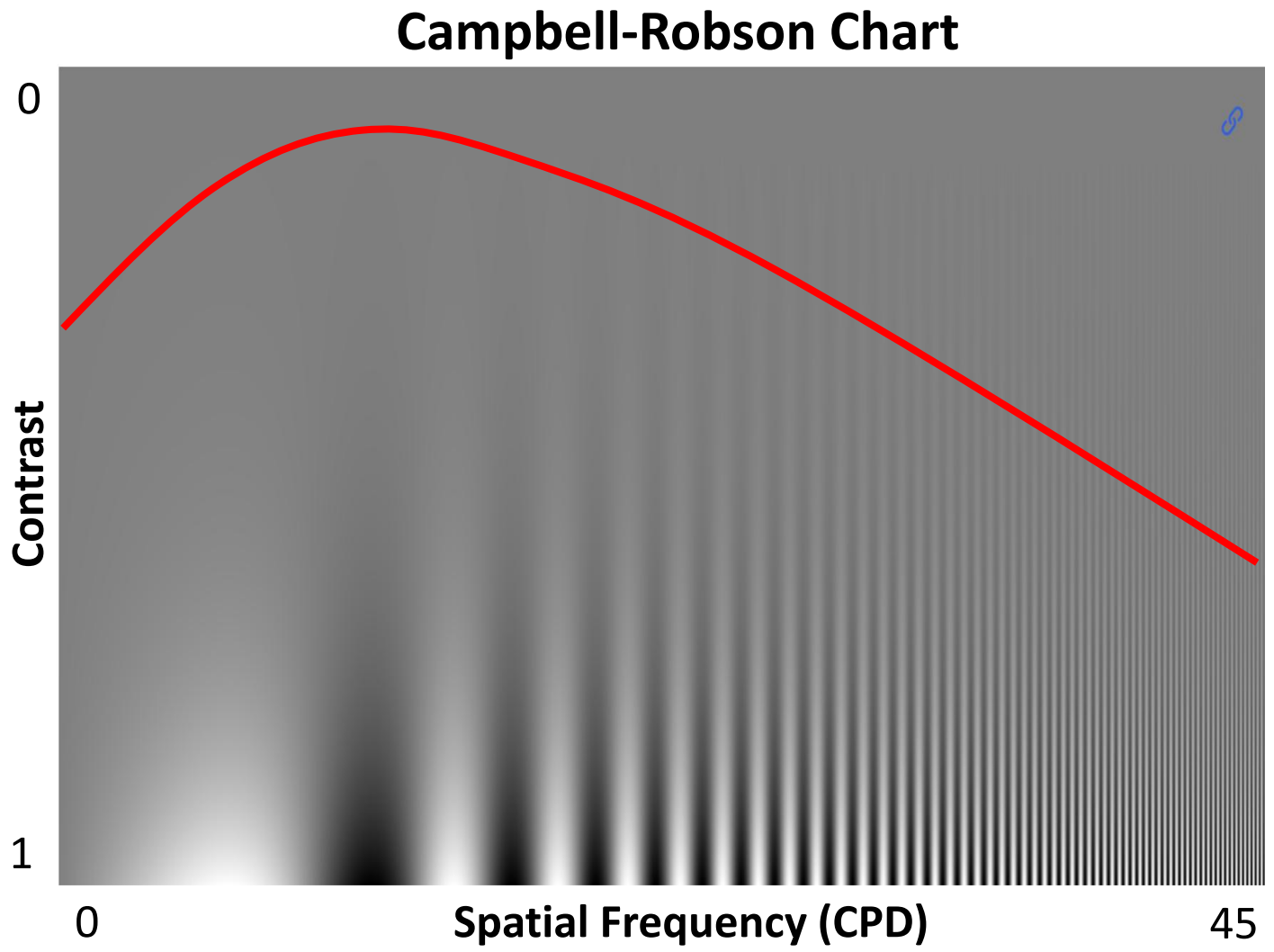
UV-Only

Spatial Frequency Sensitivity

- **Spatial Frequency**
  - How quickly image intensity changes across space
  - **Low Spatial Frequency**: slow changes over a large area, such as smooth areas or broad shapes (e.g., sky, skin).
  - **High Spatial Frequency**: rapid changes over a short distance, such as fine textures, thin lines, or sharp edges. (e.g., hair, fabric).
  - **CPD:** Spatial freq is measured in bright-dark cycles per degree (CPD) of visual angle
- **Contrast Sensitivity Function (CSF)**
  - Describes how much contrast (The difference in luminance between a pattern and its background) is needed at each spatial frequency.
  - Human visual system is not equally sensitive to all spatial frequencies;
    - typically higher at intermediate frequency
    - lower at very low and very high frequency

Spatial Frequency Sensitivity

- **Peak Sensitivity**
  - occurs in a very narrow band of mid-low freq.
- **High-Frequency Fall-off**
  - sensitivity decreases rapidly for very high-freq.
- **Low-frequency Drop-off**
  - Sensitivity decreases for very broad, smooth area
- **Perceptual Trash**
  - Above the red line, spatial details can no longer be resolved.



Visual Masking - Spatial

- **Spatial Masking**
  - occurs when surrounding texture further reduces that sensitivity. Masking is particularly strong in highly textured regions such as grass, gravel, etc.
  - Why? Strong edges and textures already generate substantial local visual responses. Small additional errors become harder to distinguish from the existing structure.
- **Compression Implication**
  - **Smooth regions:** artifacts are easier to detect
  - **Textured regions:** artifacts are more easily masked
  - More texture → higher distortion tolerance
  - Perceptual coding reallocates bits from where errors are hidden to where they are visible

Visual Masking - Temporal

- **Temporal Masking**
  - occurs when rapid temporal changes reduce the HVS sensitivity to nearby visual distortions. → Distortions that would be visible in a static scene may become harder to perceive during motion, flicker, or abrupt scene changes
- **Forward & Backward Masking**
  - Motion masking: Distortion occurring during strong motion may be less noticeable
  - Forward masking: a strong motion event reduces sensitivity to stimulus that occur shortly after it.
  - Backward masking: a later strong motion event can reduce the visibility of a stimulus presented shortly before it.
  - Fw/Bw masking effects typically occur over relatively short temporal intervals and depend strongly on stimulus strength, duration, and viewing conditions.
- **Application in Video Coding**
  - Perceptual encoders can allocate fewer bits or stronger quantization to regions/times where temporal activity masks distortion

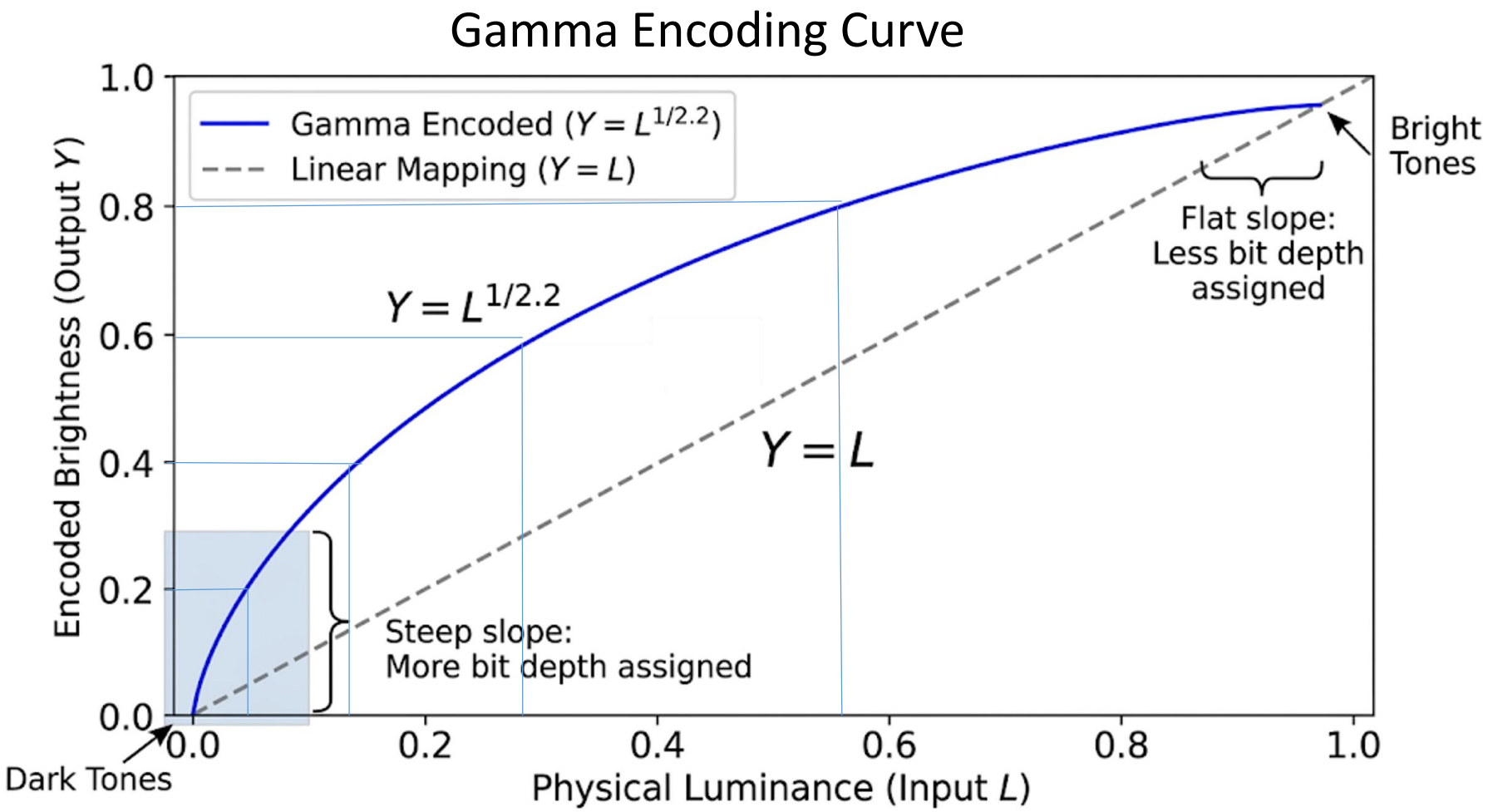
- **Human Vision System (HVS)**
  - **Part I — Fundamental Properties of Human Vision**
    - Nonlinear Brightness Perception
    - Luminance vs. Chroma Sensitivities
    - Spatial Frequency Sensitivity
    - Visual Masking
  - **Part II — From Human Vision to Perceptual Coding**
    - Gamma Encoding — Exploiting Perceptual Nonlinearity
    - Chroma Sub-sampling — Exploiting Luminance vs. Chroma Sensitivities
    - How Compression Exploits the HVS

1. Gamma Correction - Exploiting Perceptual Nonlinearity

- **Gamma Correction**
  - Digital systems use Gamma to allocate more "bit depth" to dark tones
  - It is performed right after the sensor captures light, before saving as a digital YCbCr file.
- **How does it do**
  - Physical light is linear, but human perception is non-linear.
  - To encode light into an 8-bit (256 levels) per-channel container, we apply **non-uniform sampling**: allocating a higher density of levels to dark regions (where the eye is sensitive) and a lower density to bright regions.
- **Result:** To fit a high-dynamic-range world into a low-bit-depth (8-bit) container.

  8-bit Video with Gamma looks as good to humans as 14-bit Linear Video.

Gamma Encoding

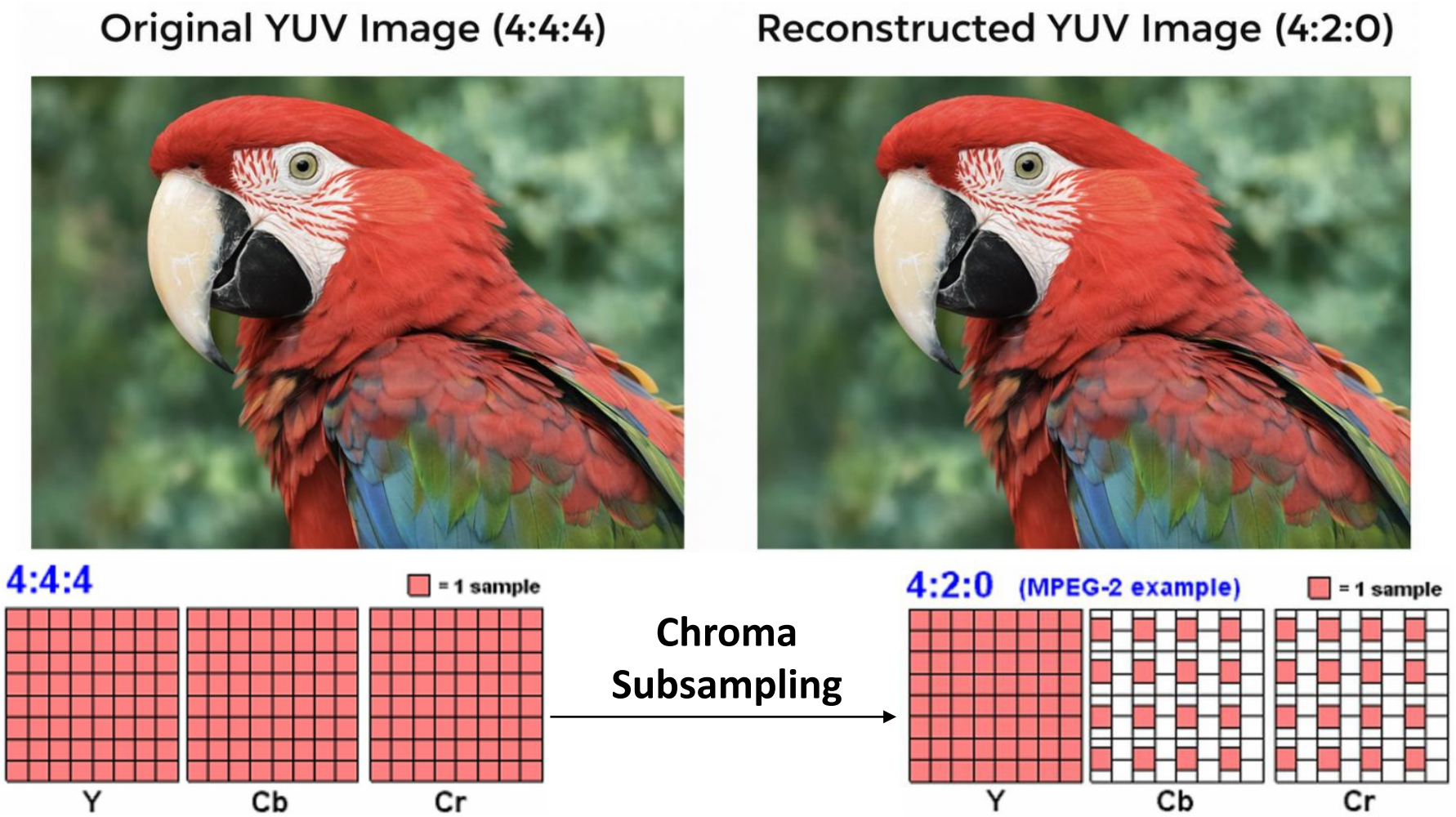


Gamma Encoding and Correction

- **From Lens to Eye**
  - **Gamma Encoding** (OETF)
    - Camera side: Light → Signal (Optical to Digital)
    - Converts **Linear Light** to **Non-linear Signal**.
    - Optimizes the 8-bit/10-bit "container" by prioritizing dark tones
    - Data processing/storing/compression 8-bit per channel digital data
  - **Gamma Correction** (EOTF)
    - Display side: Signal → Light (Digital to Optical)
    - Converts **Non-linear Signal** back to **Linear Light**.
    - The Display "undo" the Gamma curve to restore the original physical contrast intended by the creator.

2. Exploiting Chroma Sensitivity

- **Chroma Subsampling**



HVS-Summary

By understanding the HVS, we learn that we don't need to perfect every pixel—we only need to perfect the pixels the human brain actually cares about.

HVS Property	Domain	What hides the noise?	Compression Implication
Luminance Sensitivity	pixel Intensity	Extremely dark or bright areas	**Gamma:** Represent luminance according to perceptual sensitivity
Chroma Sensitivity	Color	Less sensitive to **fine chroma detail**.	Reduce chroma spatial resolution
Spatial-Freq Sensitivity	Frequency	Sensitivity varies with **spatial frequency**.	Spend fewer bits on less-visible frequency components.
Spatial Masking	Spatial Context	**Texture and edges** reduce local sensitivity.	Allow more distortion in highly textured regions
Temporal Masking	Time	**Motion and temporal activity** reduce sensitivity.	Allow more distortion during strong temporal activity

Transform Coding

- Transform Coding as Coordinate Rotation
- 1D DCT (Discrete Cosine Transform)
- 2D DCT
- Advanced Topic1: Transform Block Size
- Advanced Topic2: Integer Transform
- Advanced Topic2: DFT vs DCT

Transform Coding

- **2D Pixel Space**
  - Represent two adjacent pixel values as a 2D signal vector $$\mathbf{X} = \begin{bmatrix} P_1 \\ P_2 \end{bmatrix}$$
  - Each adjacent pixel pair forms a point in this 2D vector space
  - In natural images, adjacent pixels are highly correlated:

$$P_1 \approx P_2$$

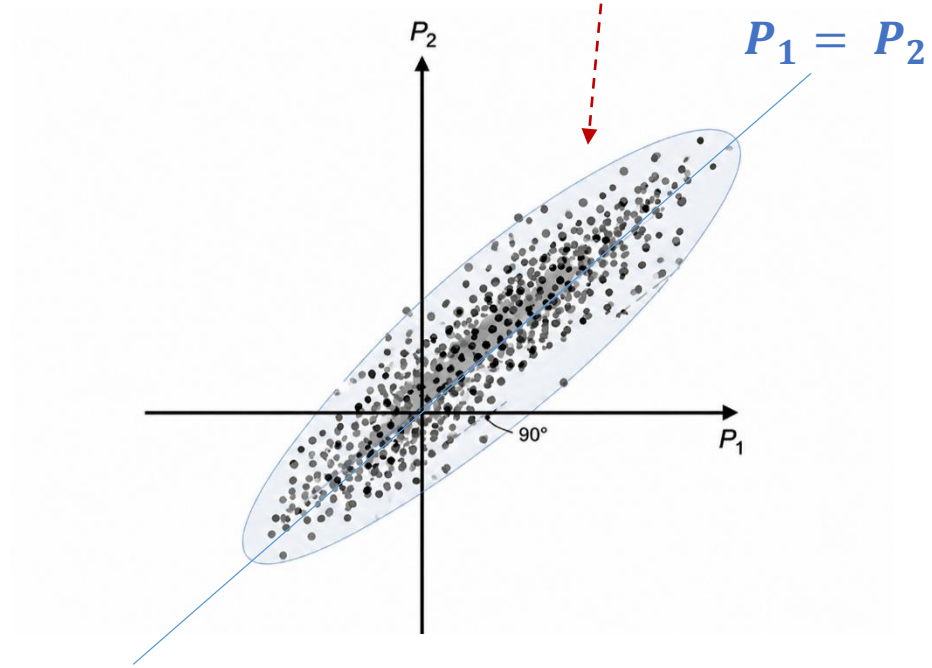
→ most data points cluster near the diagonal line

$$P_1 = P_2$$

  - Signal energy is concentrated along a correlated direction.

50 51 52 50 …
51 51 52 53 …

→ (50, 51), (52, 50) …
(51, 51), (52, 53) …



The coordinate axes are not aligned with the data distribution. → **Large variations along both P1 and P2.**

Energy is NOT compact.

Transform Coding as Coordinate Rotation

- Transform coding perform a **change of basis**, or geometrically, a rotation of coordinate axes

- **Choosing a new basis**

  e.g., Instead of describing the signal using:

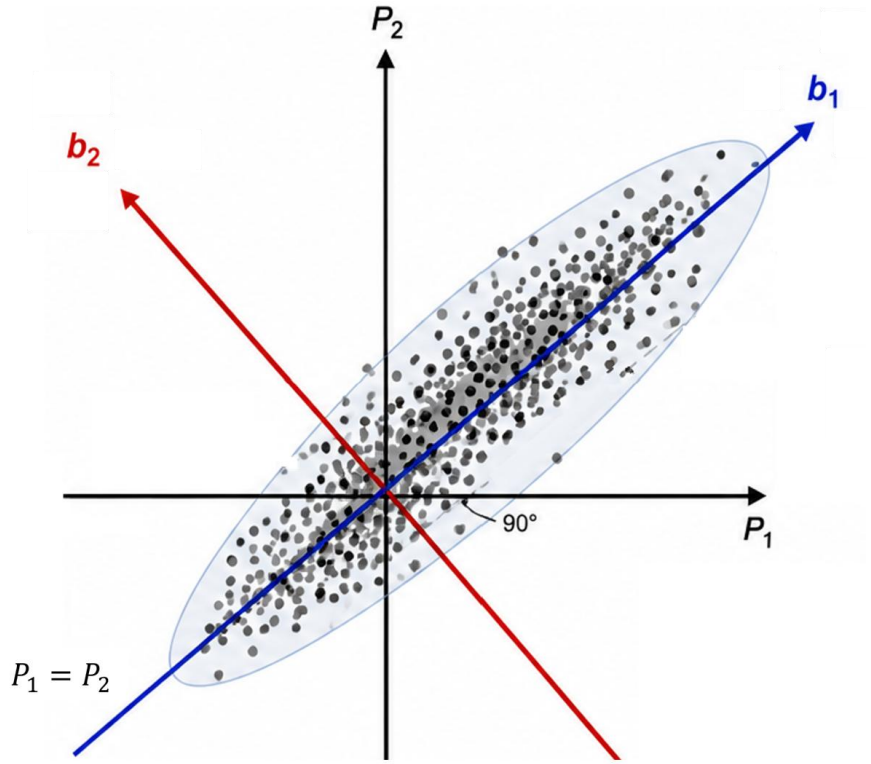
  - **P1, P2**

  we rotate coordinate system to align with data distribution

  - **dominant direction b1**
  - **variation direction b2**

  That is, the new basis:

$$\mathbf{b}_1 = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\1\end{bmatrix} \quad \mathbf{b}_2 = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\-1\end{bmatrix}$$



Large variation along b1, small variation along b2.

Energy is compact.

Transform Coding as Coordinate Rotation

- **Transform: Project on to basis vectors**
  - Projection x = (P_1, P_2) onto basis b1

$$c_1 = \mathbf{b}_1^T \mathbf{x} = \frac{1}{\sqrt{2}}(1,1) \cdot (P_1, P_2) = \frac{1}{\sqrt{2}}(P_1 + P_2)$$

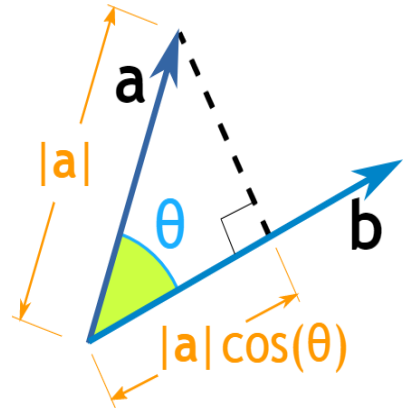
  - Projection x onto basis b2

$$c_2 = \mathbf{b}_2^T \mathbf{x} = \frac{1}{\sqrt{2}}(1,-1) \cdot (P_1, P_2) = \frac{1}{\sqrt{2}}(P_1 - P_2)$$

  - c_1: **projection coefficient** on b1
    (large values/high energy)
  - c_2: **projection coefficient** on b2
    (close to zero/low energy)

Dot Product (Inner Product)

$$\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i = \|\mathbf{a}\| \|\mathbf{b}\| \cos(\theta)$$



The basis vector is unit vector (||b|| = 1)
→ The result is the projection coefficient.

- For natural images → most energy is concentrated in c1 (along b1)
  only little energy remains in c2 (along b2)

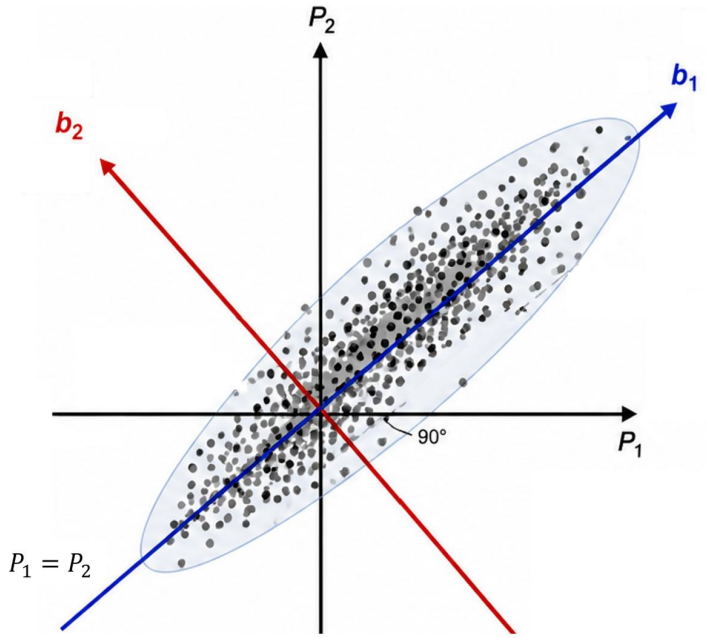
Transform Coding as Coordinate Rotation

- **Inverse Transform: Reconstructing the signal**
  - The coefficients c1, c2 indicate how strongly the signal matches a basis pattern.
  - After obtaining coefficients:
    - $\boldsymbol{c_1, c_2}$

    the signal can be reconstructed by: $x = c_1\boldsymbol{b_1} + c_2\boldsymbol{b_2} = (P_1, P_2)$
  - **Geometric interpretation**
    - Reconstruction = Weighted combination of basis patterns

- Transform vs Inverse Transform
  - **Transform:** Image → Coefficient ➔ how much of each basis pattern exits in image?
  - **Inverse Transform:** Coefficients → Image ➔ combine weighted basis patterns to reconstruct the image

Transform Coding



- **Energy Compaction**
  - In natural images:
    - $\boldsymbol{P_1 \approx P_2}$
  - Therefore
    - Most energy in $\boldsymbol{c_1}$ (projected on b1)
    - $\boldsymbol{c_2 \approx 0}$ (projected on b2)
  - Most important information is concentrated into a few coefficients.

→ This is the fundamental reason transform coding enables compression.

A transform rotates the coordinate system so that the data distribution aligns with the new axes, concentrating the "energy" (variance) on the primary/few axis.

→ Transform = change of basis

Compression = most coefficients become small

1D DCT (Discrete Cosine Transform)

- **8-point DCT**

$$C[k] = \alpha(k) \sum_{n=0}^{7} x[n] \cos\left(\frac{\pi(2n+1)k}{16}\right)$$

DCT coeff. Basis vector 8 pixel values form a 8-dim vector

$$C[k] = b_k^T x \qquad x = [x_0, x_1, \ldots, x_7]$$

- **Basis vectors** :

$$b_k[n] = \alpha(k) \cos\left(\frac{\pi(2n+1)k}{16}\right)$$

k: basis index, k: 0-7
n: basis component index, n=0-7
$\alpha(k)$: normalization function (to become unit vector)

- k=0 → $b_0 = \frac{1}{\sqrt{8}}[1,1,1,1,1,1,1,1]$ ➔ DC

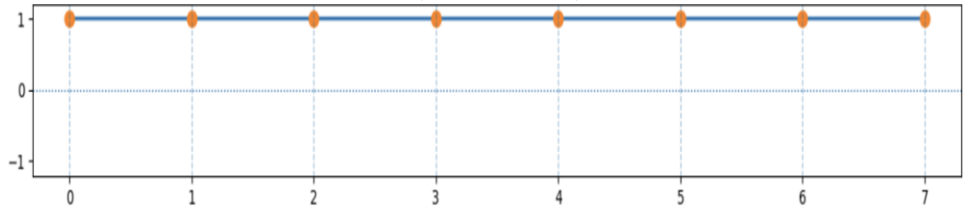
```
b0=[0.354,  0.354,  0.354,  0.354,  0.354,  0.354,  0.354,  0.354]
b1=[0.490,  0.416,  0.278,  0.098, -0.098, -0.278, -0.416, -0.490]
b2=[0.462,  0.191, -0.191, -0.462, -0.462, -0.191,  0.191,  0.462]
b3=[0.416, -0.098, -0.490, -0.278,  0.278,  0.490,  0.098, -0.416]
b4=[0.354, -0.354, -0.354,  0.354,  0.354, -0.354, -0.354,  0.354]
b5=[0.278, -0.490,  0.098,  0.416, -0.416, -0.098,  0.490, -0.278]
b6=[0.191, -0.462,  0.462, -0.191, -0.191,  0.462, -0.462,  0.191]
b7=[0.098, -0.278,  0.416, -0.490,  0.490, -0.416,  0.278, -0.098]
```

8-point 1D DCT

$$\cos\left(\frac{(2n+1)k\pi}{16}\right)$$

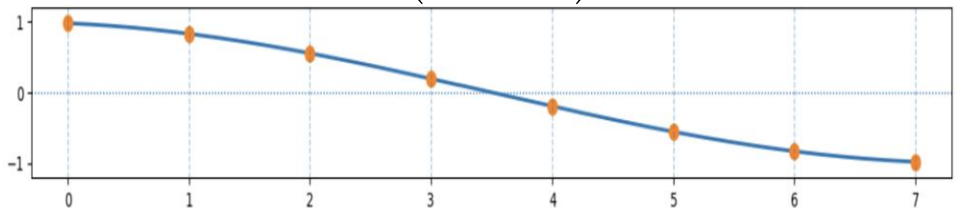
k=0

$$\cos\left(\frac{(2n+1)0\pi}{16}\right)$$



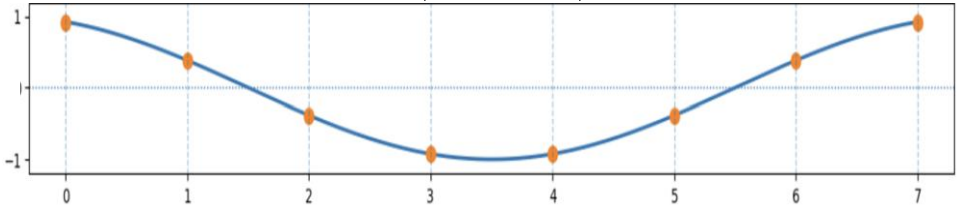
k=1

$$\cos\left(\frac{(2n+1)1\pi}{16}\right)$$



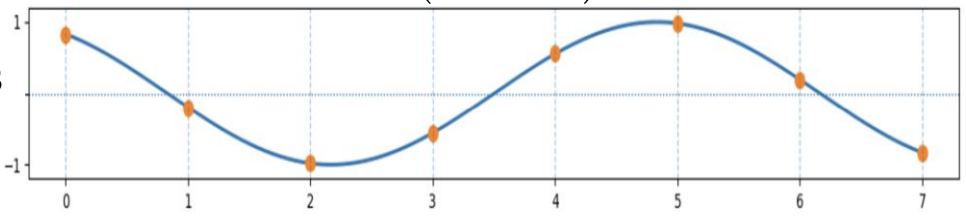
k=2

$$\cos\left(\frac{(2n+1)2\pi}{16}\right)$$



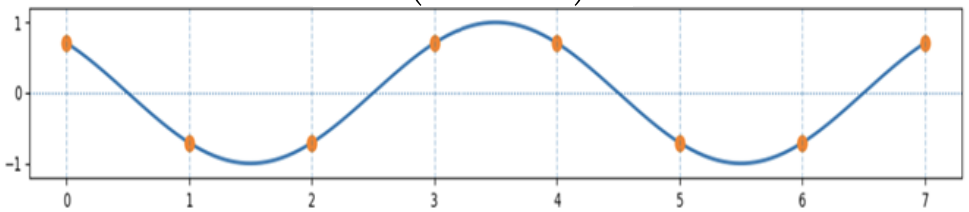
k=3

$$\cos\left(\frac{(2n+1)3\pi}{16}\right)$$



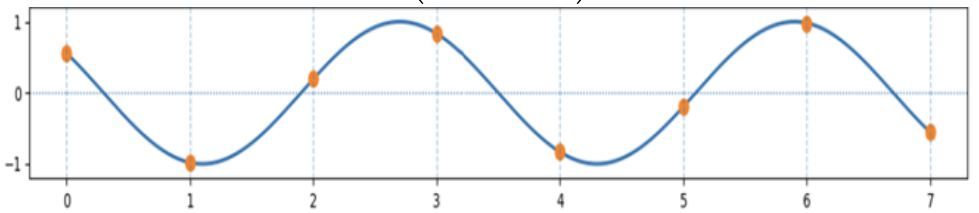
k=4

$$\cos\left(\frac{(2n+1)4\pi}{16}\right)$$



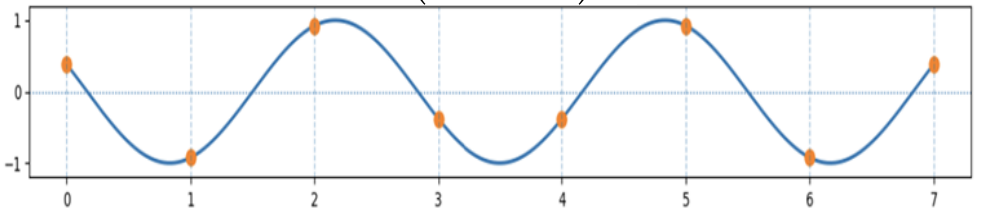
k=5

$$\cos\left(\frac{(2n+1)5\pi}{16}\right)$$



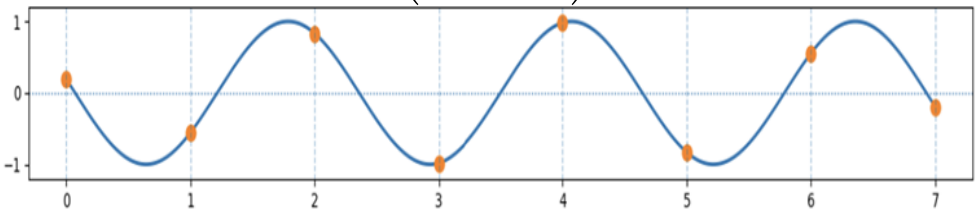
k=6

$$\cos\left(\frac{(2n+1)6\pi}{16}\right)$$



k=7

$$\cos\left(\frac{(2n+1)7\pi}{16}\right)$$

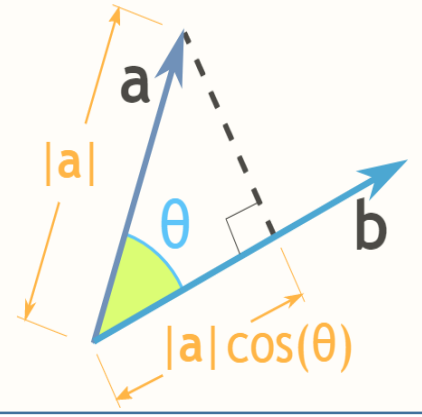


1D DCT

- **Interpretation of DCT Coefficients**
  - DCT coefficient = projection onto basis direction
  - Basis vectors are all unit vectors → inner product (dot product) can be the projection result.

Inner product

$$\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i = \|\mathbf{a}\| \|\mathbf{b}\| \cos(\theta)$$



  - **Geometric meaning of DCT coefficients**
    - C[k] : the projection of signal x onto the basis direction b_k
    - measuring the similarity between the signal and each basis vector

1D DCT

- **Case 1: Low-Frequency Signal** (Smooth)
  - **Signal** x = [10, 11, 10, 10, 11, 10, 11, 10] (The signal is relatively flat with a high average value)
    - **DC** basis: b_0 = [1,1,1,1,1,1,1,1]
      - C[0] = $b_0^T x \approx 83$ large coefficient → The signal has a strong constant background
    - **High-freq** basis: b_7 = [1,−1,1,−1,1,−1,1,−1]
      - C[7] = $b_7^T x \approx 1$ small coefficient → almost no rapid oscillation in this signal.
- **Case 2: High-Frequency Signal** (Rapid Changes)
  - **Signal** x = [10,− 11, 10,− 10, 11,− 10, 11,− 10] (The signal fluctuates rapidly.)
    - **DC** basis (b_0) :
      - C[0] = $b_0^T x \approx 1$ small coefficient → The average value is near zero
    - **High-freq** basis (b_7):
      - C[7] = $b_7^T x \approx 83$ large coefficient → The signal highly aligns with the alternating basis b_7

1D DCT

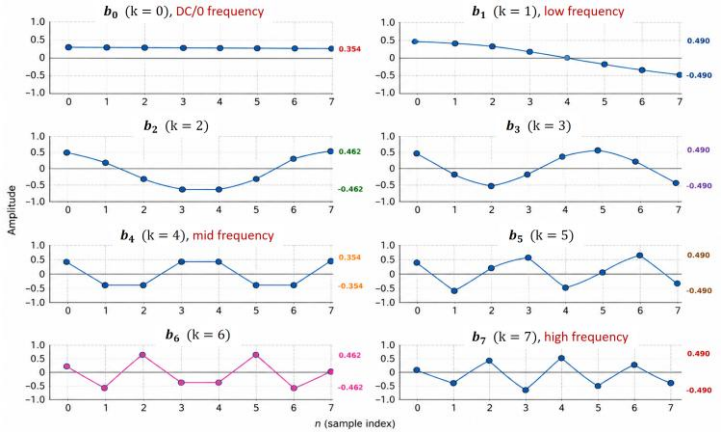
- **Frequency Basis as Vector Directions**
  - Each frequency basis function represents a vector direction in high-dimensional space
    - e.g., DC basis: [1,1,1,1,1,1,1,1], High-frequency basis: [1, -1, 1, -1, 1, -1, 1, -1]
- **Inner Product = Similarity Measure**
  - The coefficient measures the "similarity " between the signal and the frequency pattern.
  - Large C[k] → strong presence of the k-th frequency pattern.
- **Frequency Transform = change of basis**
  - From pixel space (spatial domain) → To frequency pattern space (frequency domain)
  - From describing pixel value → To how much of each frequency pattern exists in signal
- **Energy Compaction**
  - Natural images contain most "energy" in DC and low-frequency coefficients.
  - High-frequency coefficients are usually small ➔ This property enables efficient compression.

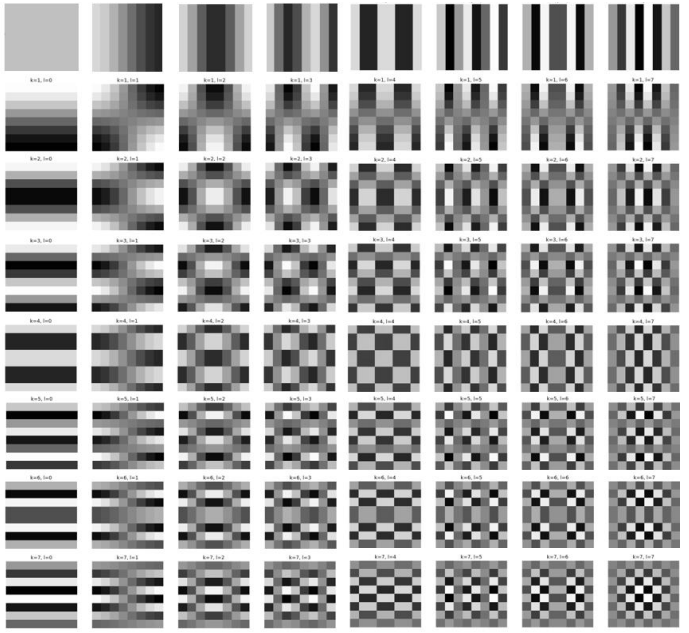
From 1D DCT to 2D DCT (8-point → 8×8)

- **1D DCT** (8-point signal)
  - Analyzes a single row or column of pixels (1D signal)
  - Input: 1D vector $x \in R^8$
  - Output: 8 frequency coefficients
- **2D DCT:**
  - Analyzes an 8x8 block of pixels simultaneously.
  - Input: 2D signal (image block) $x \in R^{8\times8}$
  - Output: 64 frequency coefficients C[*u*, *v*] *u*,*v* = 0, …,7
  - Process: Change of Basis (Projection onto 64 2D-patterns).

$$b_{u,v}(i,j) = \cos\left(\frac{(2i+1)u\pi}{16}\right)\cos\left(\frac{(2j+1)v\pi}{16}\right)$$

  - u: horizontal frequency index (0~7)
  - v: vertical frequency index (0~7)





2D DCT Basis Construction (Outer Product)

- **From 1D to 2D Spatial Frequency**
  - A 2D DCT basis image is generated by the **outer product** of a 1D vertical basis vector and a 1D horizontal basis vector:

$$B_{u,v}(x,y) = b_u(x)b_v(y)$$

The vertical pattern and horizontal pattern multiply together to form a 2D pattern.

  - **Example**

1D vertical basis

$$\mathbf{u} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

1D horizontal basis

$\mathbf{v}$ = [1 -1 1 -1]

2D basis

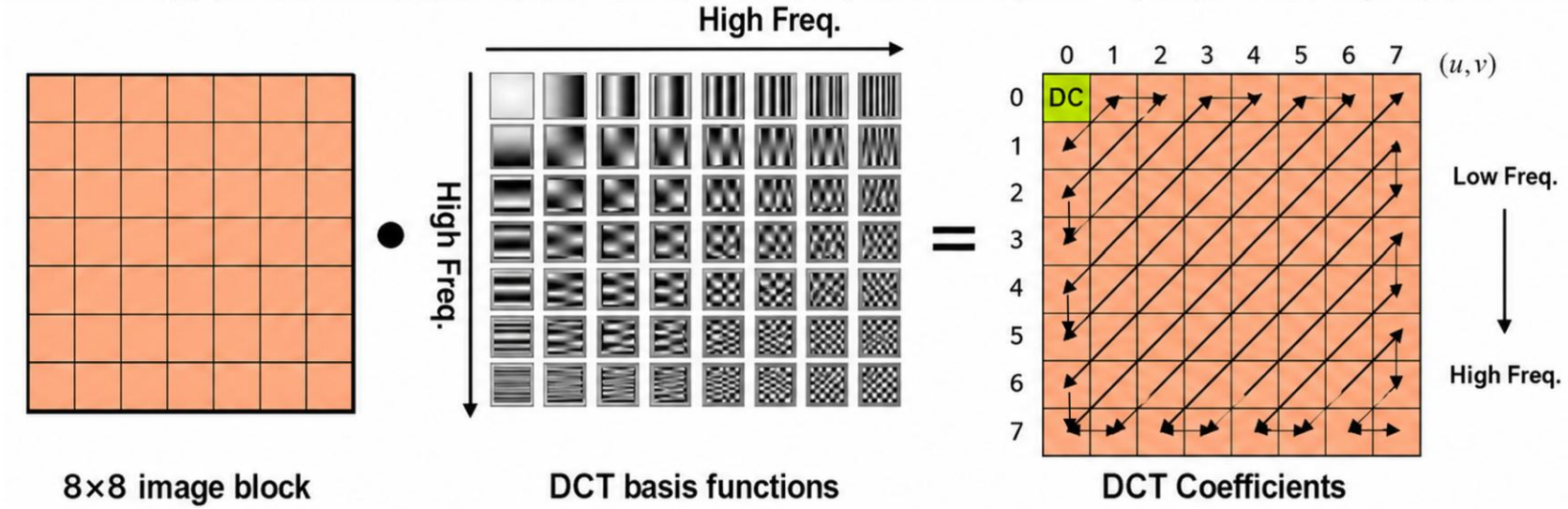
$$\mathbf{u}\mathbf{v}^T = \begin{bmatrix} 1 & -1 & 1 & -1 \\ 1 & -1 & 1 & -1 \\ 1 & -1 & 1 & -1 \\ 1 & -1 & 1 & -1 \end{bmatrix}$$

- **Direct 2D Projection**
  - The 2D DCT can be expressed either in explicit cosine form or compact basis form:

$$C(u,v) = \frac{\alpha(u)\alpha(v)}{4} \sum_{x=0}^{7} \sum_{y=0}^{7} f(x,y) \cos \frac{(2x+1)u\pi}{16} \cos \frac{(2y+1)v\pi}{16} = \sum_{x=0}^{7} \sum_{y=0}^{7} f(x,y) b_u(x) b_v(y)$$

2D DCT (Forward Transform)

- **2D DCT Process** $$C(u,v) = \sum_{x=0}^{7}\sum_{y=0}^{7} f(x,y)b_u(x)b_v(y)$$
  - **2D inner product:** Projects the entire 8x8 image block onto a 2D-basis pattern → 1 coefficient
  - Coefficient: Quantifies how much of that specific 2D frequency pattern exists in the 8x8 image block.
    - 64 basis patterns → 64 coefficients (1 **DC** coefficient , 63 **AC** coefficients)
    - Top-Left Corner C(0,0): DC Coefficient. Average brightness of the entire 8x8 block.
    - Bottom-Right Corner C(7,7): The highest AC frequency. .

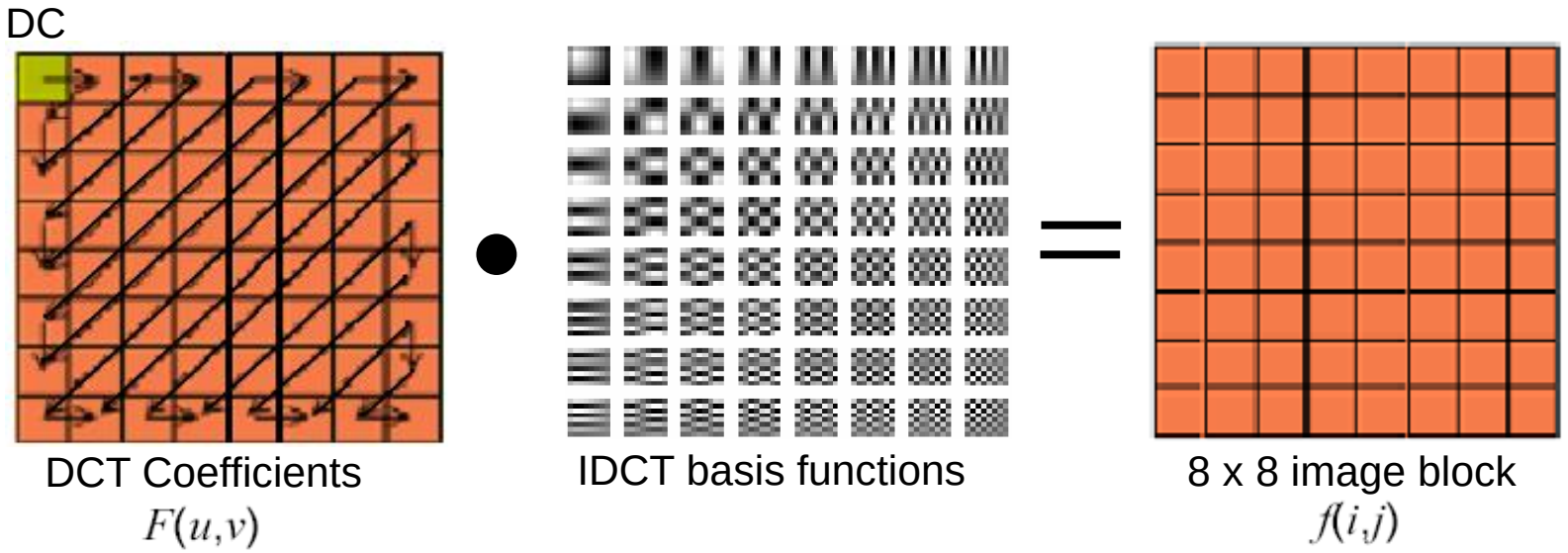


Inverse 2D DCT (Image Reconstruction)

- **IDCT Process**

$$f(x,y) = \sum_{u=0}^{7}\sum_{v=0}^{7} C(u,v)b_u(x)b_v(y)$$

  - Reconstructing Pixels: 64 frequency coefficients → 64 pixel values
  - **Linear Combination of all basis patterns**: Any 8x8 pixel block is mathematically reconstructed by adding together all 64 basis images, using each corresponding $C(u, v)$ as a **weight**.
  - Inverse DCT: If no coefficients are altered, the reconstruction is mathematically lossless.



2D DCT: Mathematical Symmetry

- **Forward 2D DCT** (Pixels to Frequencies)

$$C(u,v) = \sum_{x=0}^{7}\sum_{y=0}^{7} f(x,y)b_u(x)b_v(y)$$

  - We loop through all pixels f(x, y) to find the weight of a single frequency C(u, v)

- **Inverse 2D DCT** (Frequencies to Pixels)

$$f(x,y) = \sum_{u=0}^{7}\sum_{v=0}^{7} C(u,v)b_u(x)b_v(y)$$

  - We loop through all frequencies (u, v) and scale each basis pattern by its weight C(u, v) to rebuild a single pixel f(x, y)

Note: The two formulas are nearly identical! The only difference is what domain you are summing over to reach the other domain.

From 2D Projection to Separable Computation

- **The Computational Bottleneck**
  - Performing the direct 2D DCT projection requires nested loops over every pixel for every frequency pattern

$$C(u,v) = \sum_{x=0}^{7} \sum_{y=0}^{7} f(x,y) b_u(x) b_v(y)$$

One coefficient → O(N^2)

  - Direct 2D DCT requires **O(N^4)** operations to compute all N^2 coefficients — inefficient.
- **Solution: Separability**
  - Because the 2D basis function is simply a product of two 1D basis functions ($b_u(x)b_v(y)$) , we can mathematically split the double summation

$$C(u,v) = \sum_{x=0}^{7} b_u(x) \underbrace{\left[\sum_{y=0}^{7} f(x,y) b_v(y) \right]}$$

1D DCT along y-direction (columns)

2D DCT Separable Computation

- **2D DCT as two consecutive 1D DCTs**
  - **Step1: Horizontal 1D DCT** (row-by-row)
    - Apply 1D DCT to every row of the input block F

$$\mathbf{G} = \boldsymbol{F} \cdot \boldsymbol{B}^{\boldsymbol{T}}$$

(B: 1D-DCT basis matrix, G: intermediate result matrix)

  - **Step2: Vertical 1D DCT** (column-by-column)
    - Apply 1D DCT to every column of the matrix G

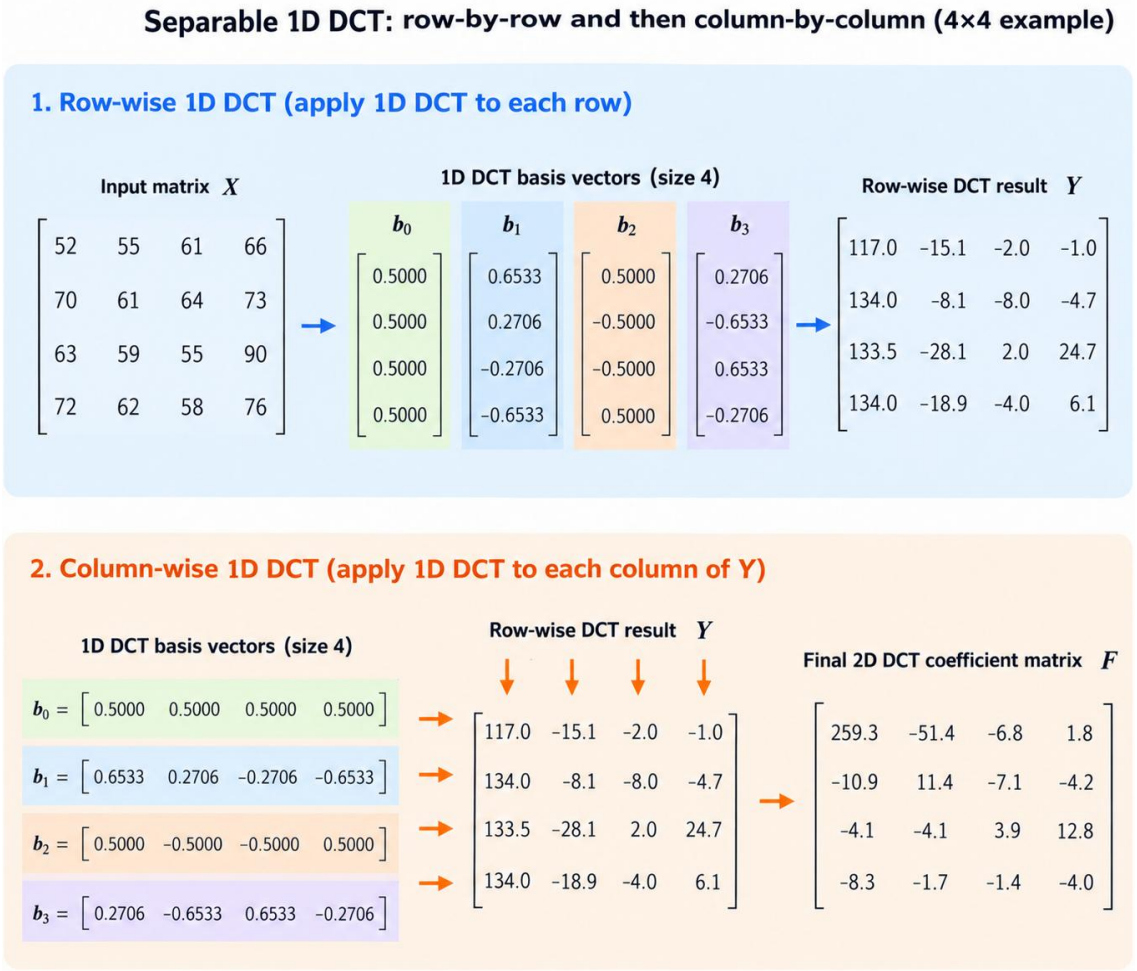
$$\mathbf{C} = \boldsymbol{B} \cdot \boldsymbol{G}$$

Final 2D DCT: $\mathbf{C} = \boldsymbol{BFB}^{\boldsymbol{T}}$

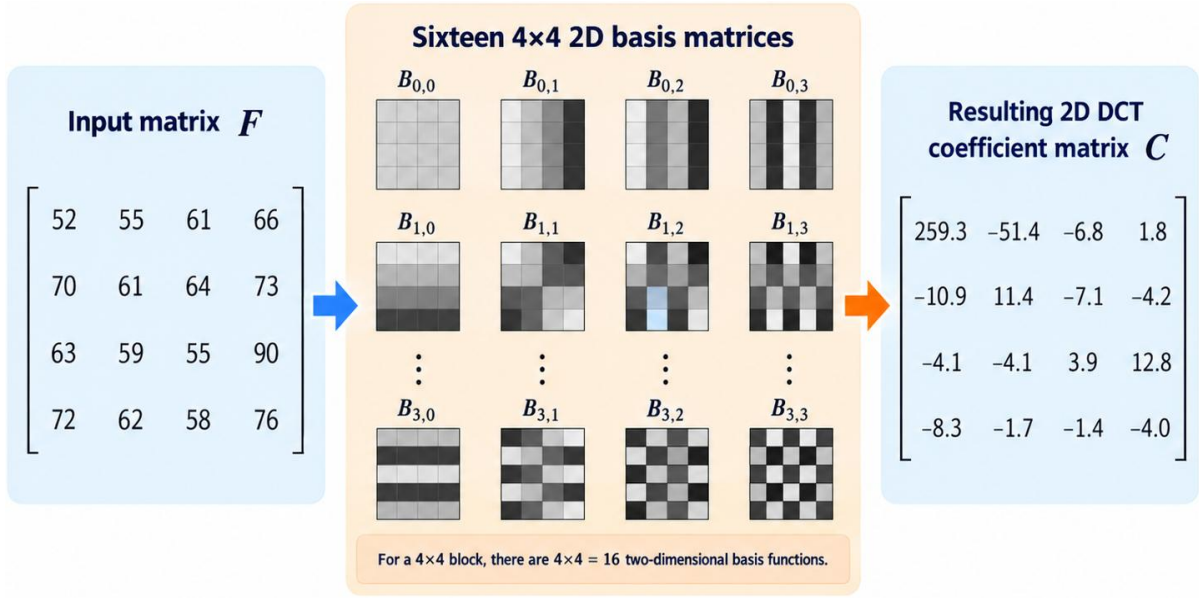
C: NxN frequency coefficients)

- **Why Separable Transform Matters**
  - Direct 2D projection complexity: $O(N^4)$
  - Two sequential 1D transforms: $O(N^3) + O(N^3) = O(N^3)$
    - Much lower computation cost, reduced memory bandwidth
  - Widely used in compression standards like JPEG, H.264/HEVC/VVC

Two consecutive 1D DCTs



Direct 2D DCT



2D DCT Energy

- **Energy Compaction**
  - 2D DCT: Analyzes an 8x8 block of pixels simultaneously.
  - Pixels in natural images are highly correlated with their neighbors.
  - After performing 2D DCT, most of the "important" information (signal energy) is packed into the **top-left** low-frequency coefficients. The high-frequency coefficients (bottom-right) are often very small or zero.
  - We can discard them with minimal loss in visual quality.

52	55	61	66	70	61	64	73
63	59	66	90	109	85	69	72
62	59	68	113	144	104	66	73
63	58	71	122	154	106	70	69
67	61	68	104	126	88	68	70
79	65	60	70	77	68	58	75
85	71	64	59	55	61	65	83
87	79	69	68	65	76	78	94

Pixel domain (Pixel value)

DC coefficient

609	-29	-62	25	55	-20	-1	3
7	-21	-62	9	11	-7	-6	6
-46	8	77	-25	-30	10	7	-5
-50	13	35	-15	-9	6	0	3
11	-8	-13	-2	-1	1	-4	1
-10	1	3	-3	-1	0	2	-1
-4	-1	2	-1	2	-3	1	-2
-1	-1	-1	-2	-1	-1	0	-1

Frequency domain (Coefficient)

Advanced Topic1: Transform Block Size

DCT Block Size Selection

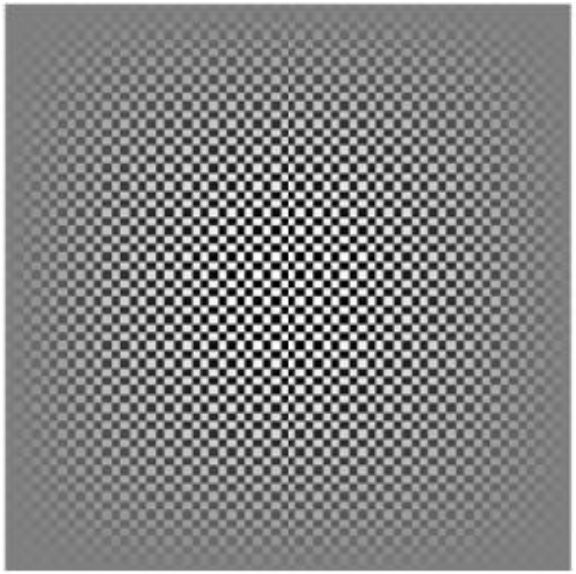
- **The Baseline**
  - In the absence of quantization, the DCT is perfectly reversible.
  - Any orthogonal DCT block size (from 4×4 to 64×64) can theoretically achieve perfect reconstruction
- **The Real-World Dilemma**
  - In practical video coding, performance is governed by a three-way trade-off:
    1. Energy Compaction
       - How efficiently signal energy is concentrated into a small number of coefficients
    2. Quantization Artifact Behavior
       - How compression artifacts appear, spread, and become visually noticeable after lossy quantization
    3. Computational Complexity
       - The processing, memory, and hardware cost of implementing larger transforms

Analyzing DCT Transform Block Sizes

- **64x64 DCT** (a single block) **vs 8x8 DCT** (64 small blocks)
  - **Both cases use**
    - Same total number of coefficients (4096)
    - Same cosine basis family
    - Both are perfectly invertible
  - **Fundamental Difference**
    - Spatial Support of Basis
      - 64x64 DCT → global basis functions
      - 8x8 DCT → local basis functions
  - Both cases exhibit fundamentally different behavior in compression

Example DCT basis

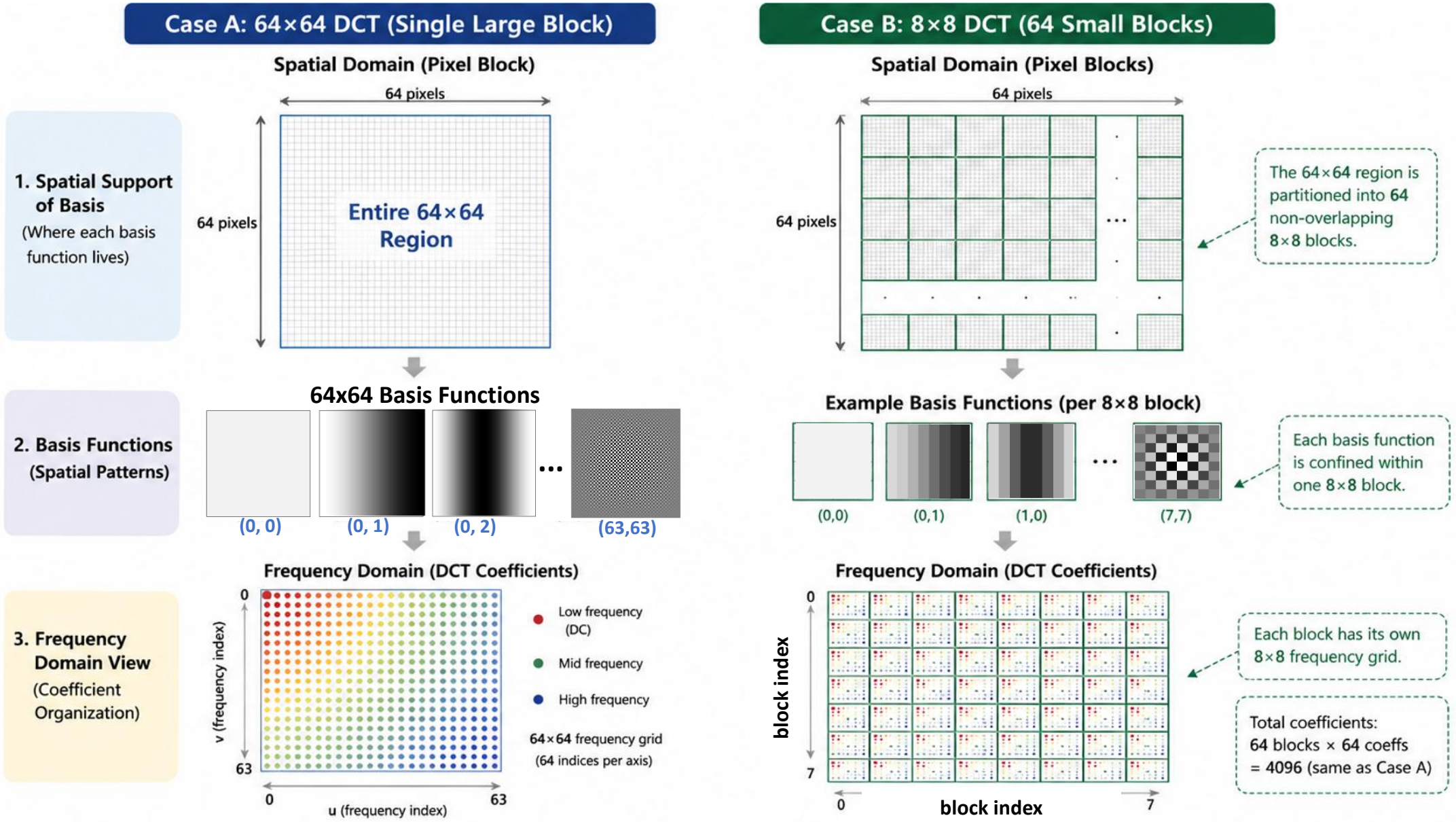
64x64 basis(63,63)



8x8 basis (7,7)



Consider a 64×64 image region with different DCT block sizes



Large DCT Transform Blocks

- **Performance of Large Transform Blocks**
  1. **Energy Compaction** (Strong in Smooth Regions)
     - Excellent at representing slowly varying signals (sky, walls, gradients)
     - Energy concentrates into few low-frequency coefficients
     - High-frequency coefficients often become near-zero after quantization
     - → Result: **high compression efficiency in smooth areas**
  2. **Quantization Error Behavior** (Weakness)
     - If an edge or texture exists, high-frequent coefficients are heavily quantized
     - Quantization error spreads across the whole block
     - → Artifact: ringing / mosquito noise
  3. **Computational Cost** (High)
     - Larger transform = higher matrix operation cost
     - More memory bandwidth + higher hardware complexity

Small DCT Transform Blocks

- **Performance of Small Transform Blocks**
  - **Energy Compaction** (Limited, but Localized)
    - Cannot exploit long-range correlation effectively
    - Smooth regions require many blocks → more DC coefficients + overhead
    - But: Works well for **high-detail / high-frequency textures**
  - **Quantization Error Behavior** (Well Contained)
    - Quantization errors stay **within small spatial region**
    - Edge artifacts do not spread far
    - → better visual localization of compression errors
  - **Computational Cost** (Low)
    - Smaller matrices → fewer operations
    - Lower memory usage and easier hardware implementation
    - Suitable for real-time / low-power systems

Large vs Small DCT Blocks

Aspect	Large Blocks	Small Blocks
Energy compaction	Excellent (smooth areas)	Moderate
Quantization error	Poor (global artifacts, one bad quantized coefficient affects entire region)	Good (localized errors, errors are confined within each block)
Compression efficiency	High (smooth images)	Lower (more overhead)
Computation	High	Low

Dynamic Block Size Allocation

- **MPEG-2 (Fixed 8×8 Blocks)**
  - A static compromise constrained by hardware and computational limitations, balancing transform efficiency and artifact control.
- **Modern Codecs (HEVC / VVC)**
  - Adapt block size dynamically according to local image characteristics:
  - Smooth Regions:
    - Large Blocks → Higher energy compaction and coding efficiency
  - Edge / Texture Regions:
    - Small Blocks → Better localization of quantization artifacts

Adaptive block partitioning enables modern codecs to optimize the tradeoff among transform efficiency, artifact containment, and computational complexity on a region -by-region basis.

Advanced Topic2: Integer Transform

The Hidden Problem of Floating-Point DCT

- **Why Floating-Point DCT is Avoided in Practice**
  - **The Irreversibility of Float:** Standard DCT basis matrix B contains cosine terms (cos), which are irrational numbers. This requires **floating-point** calculations.
  - **Hardware Inefficiency:** Floating-point units (FPUs) are slower, consume more power, and take up more silicon area in a chip compared to simple integer logic.
- **The "Floating-Point Drift" Nightmare**
  - **The Issue:** Different processor architectures (Intel, ARM, AMD) or software compilers handle floating-point rounding slightly differently
  - The result of DCT→ IDCT
    - Because of minor rounding differences, the decoded pixels won't match the original perfectly.
    - In video coding, this error **accumulates over time** across consecutive frames, leading to severe visual corruption (Drift).

The Core Idea of Integer Transform

- **How to Eliminate Floats?** **Matrix Scaling**
  - Modern coding standards replace the DCT with an **Integer Approximation**
- Example: 4×4 Transform Kernel (H.264-like)
  - A simplified DCT-like kernel can be written as:

$$B = \begin{bmatrix} 0.5 & 0.5 & 0.5 & 0.5 \\ 0.653 & 0.271 & -0.271 & -0.653 \\ 0.5 & -0.5 & -0.5 & 0.5 \\ 0.271 & -0.653 & 0.653 & -0.271 \end{bmatrix}$$

  - **Integer Approximation Process**
    1. Multiply the matrix by a scaling factor (e.g., 2^k)
    2. Apply rounding to nearest integer

    Resulting integer transform (illustrative form) ➔

$$\tilde{B} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & 1 & -1 & -2 \\ 1 & -1 & -1 & 1 \\ 1 & -2 & 2 & -1 \end{bmatrix}$$

Multiplication-Free Implementation

- **Shift-and-Add Hardware Design**
  - In digit IC design, multipliers are expensive, but adders and shifters are incredibly fast and cheap.
  - **Integer matrix example**
    - Operations become:
      - × 2 → left shift (<<1)
      - × 1 → pass-through
      - ×(-1) → sign inversion
      - ×(-2) → left shift + sign inversion

$$\tilde{B} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & 1 & -1 & -2 \\ 1 & -1 & -1 & 1 \\ 1 & -2 & 2 & -1 \end{bmatrix}$$

  - The Integer Transform allows the CPU/GPU to compute the entire spatial-to-frequency transform using only **additions** and **bit-shifts**. No floating-point math, no actual multiplication units needed!
  - This significantly reduces: hardware complexity, latency, power consumption

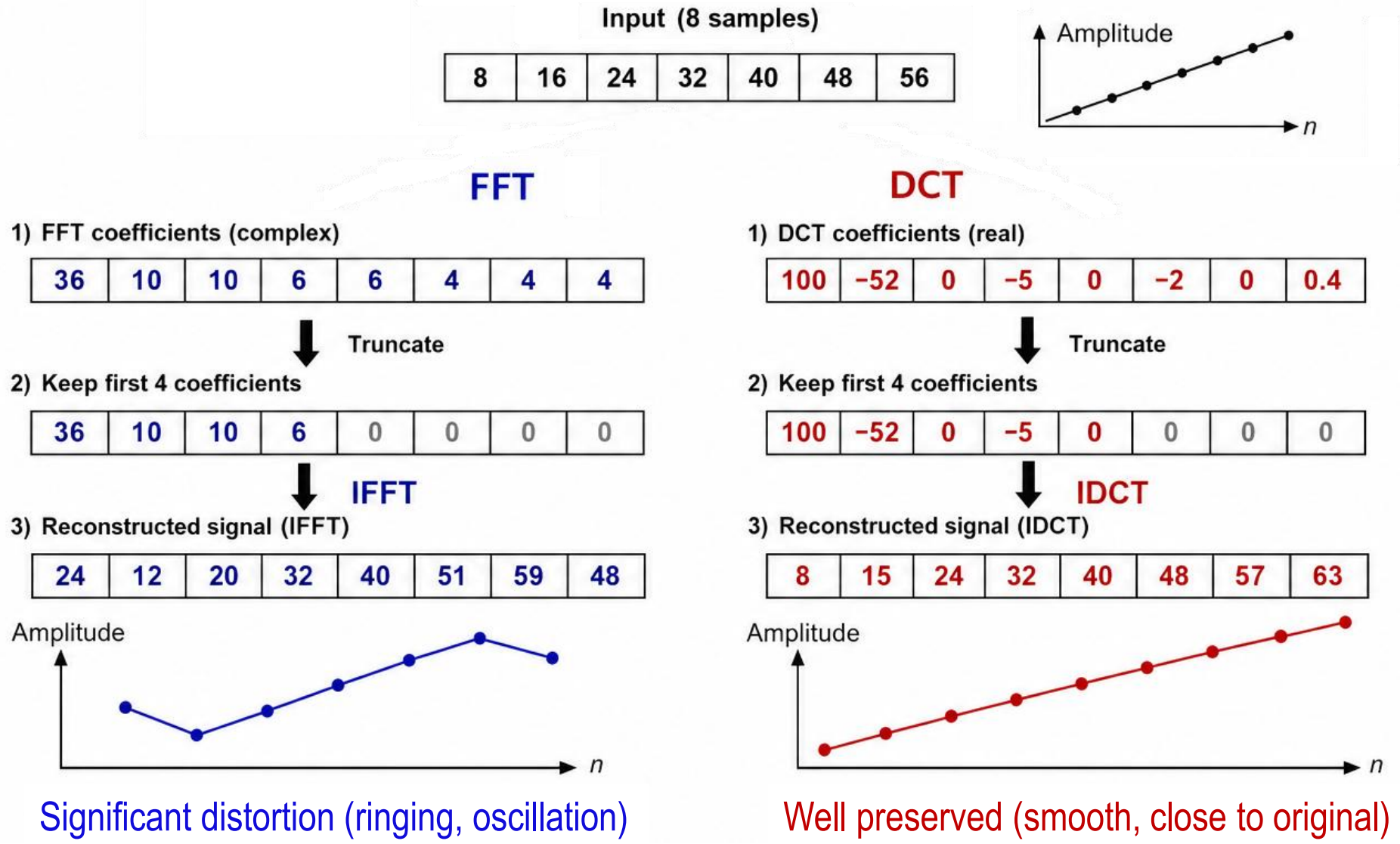
Where Does the Normalization Go?

- **Question:** If we multiplied the matrix by a constant earlier, didn't we change the energy of the image?
  - Yes, we did. The output coefficients are now scaled up.
  - The Solution: We delay the normalization and merge it into the Quantization step that follows DCT.
  - Since Quantization already involves division, we combine the transform scaling factor and the quantization step into a single look-up table (LUT).

Advanced Topic 3: DFT vs DCT

- **Why Image/Video Compression Uses DCT Instead of DFT**

DFT vs DCT: Energy Compaction



Significant distortion (ringing, oscillation)

Well preserved (smooth, close to original)

Quantization

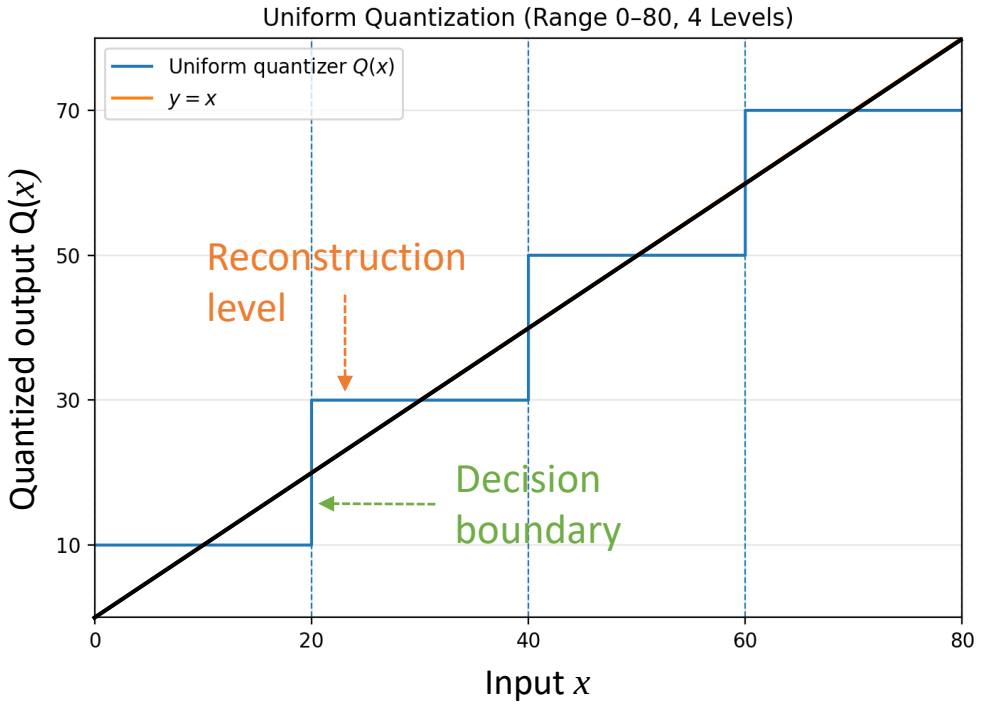
- Uniform Quantization
- Quantization Noise Model
- Rate-Distortion Theory
- Non-Uniform Quantization
- Vector Quantization

Transition to Quantization

- **The Question**
  - We have seen that the HVS is **not equally sensitive to all visual information**.
  - Some frequency components, colors, textures, or temporal conditions can tolerate more distortion than others.
  - But how can a codec actually reduce precision and save bits
- **The Solution: Quantization**
  - Quantization is the process of mapping a large set of values to a smaller set.
  - The **Quantization Step Size (Δ)** acts as a **quality–bitrate control knob**.
    - Small Δ : More levels → higher precision → lower distortion → more bits.
    - Large Δ : Fewer levels → lower precision → higher distortion → fewer bits
  - **Perceptual knowledge tells us where distortion is acceptable; quantization controls how much distortion is introduced by the representation.**

Quantization Concept

- **Quantization:** maps a continuous or high-precision input value to one of a finite set of discrete output levels.
  - **As a step function:** $Q(x) = y_i$ if $b_{i-1} < x \leq b_i$
  - **Common operations**: rounding or truncation
  - **Decision boundary:** determine which quantization interval an input belongs to
  - **Reconstruction levels:** the output values used to represent each interval.
  - **Quantization Error** : $e = Q(x) - x$
- **Role in DSP**
  - **Analog-to-Digital Conversion**: converts continuous-amplitude signals into discrete digital levels.
  - **Lossy Compression**: reduces representation precision to save bits, forming a core component of image, audio, and video codecs



Quantization - Mathematical Framework

- Quantization is a Two-Stage Model
  - **Forward Quantization (Index Assignment)**
    - Maps input x to a discrete integer index k
    - Example: $k = \left\lfloor \frac{x}{\Delta} \right\rfloor$, $k = round(\frac{x}{\Delta})$, … Δ: quantization step size
  - **Inverse Quantization (Reconstruction)**
    - Maps the index k back to an approximate value $y_k = k \cdot \Delta$
  - **Example:**

$$k = \text{round}(\frac{x}{\Delta}) \ , \Delta = 30$$

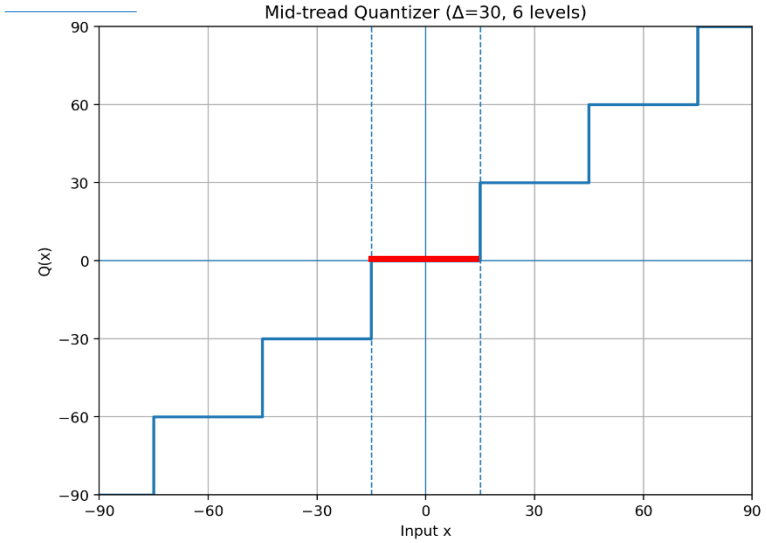
$-14 \le x < 15 \rightarrow k=0 \rightarrow y = 0$
$15 \le x < 45 \rightarrow k=1 \rightarrow y = 30$
$45 \le x < 75 \rightarrow k=2 \rightarrow y = 60$

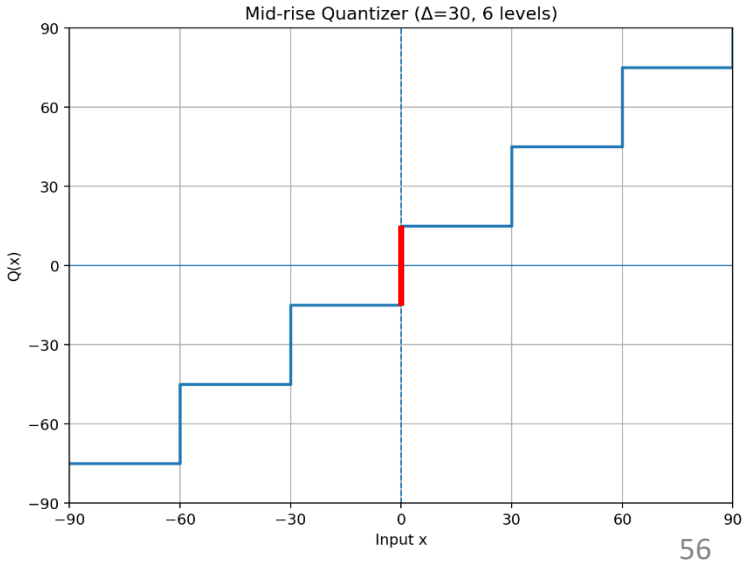
  - **Irreversibility**
    - Quantization is a many-to-one mapping; the original value cannot be recovered.

Quantization discards precision; inverse quantization only reconstructs a representative value.

Uniform Quantizers

- **Mid-Tread Quantizer** $Q(x) = \Delta Round\left(\frac{x}{\Delta}\right)$
  - **Origin**: (0, 0) lies in the middle of a horizontal "tread."
  - **Zero Level**: Includes a reconstruction level at exactly 0.
  - Useful for signals with many small values around zero, such as audio silence, prediction residuals, and transform coefficients.
- **Mid-Riser Quantizer** $Q(x) = \Delta\left(\left\lfloor \frac{x}{\Delta} \right\rfloor + \frac{1}{2}\right)$
  - **Origin**: (0, 0) lies in the middle of a vertical "riser."
  - **Zero Level**: No zero reconstruction level. Smallest outputs are $\pm 0.5\Delta$
  - Useful when zero does not need special treatment





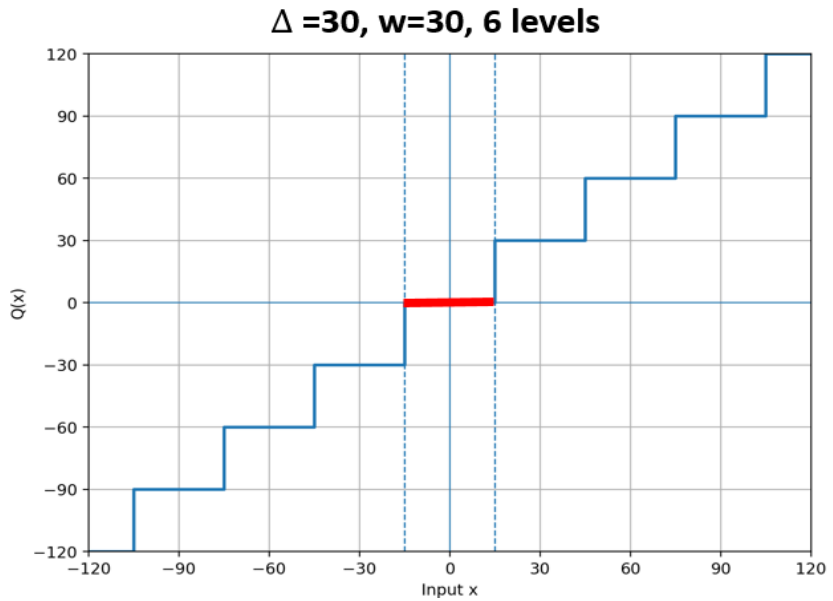
Dead-Zone Quantizers

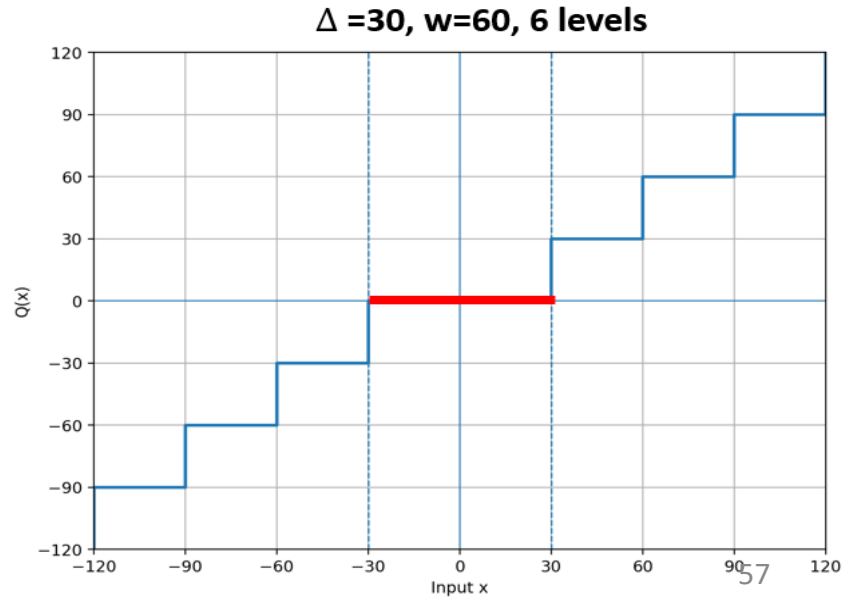
- **Dead-Zones for Noise Suppression**
  - A mid-tread variant where the center "zero" step is wider than other steps.

$$Q(x) = \begin{cases} 0, & |x| < \frac{w}{2} \\ \operatorname{sgn}(x)\,\Delta\left(\left\lfloor \frac{|x| - \frac{w}{2}}{\Delta} \right\rfloor + 1\right), & |x| \geq \frac{w}{2} \end{cases}$$

  - All input values inside the **dead zone** are mapped to zero.
  - **Compression Benefit**

    Dead-zone quantization increases the number of zero-valued coefficients. It is commonly used in transform-based image and video coding.





Quantization Noise Model

- **Additive Noise & The 6 dB Rule**
  - **Additive Noise Model**
    - Quantization error: $e = Q(x) - x$
    - **MSE** (Mean Squared Error): $\sigma_e^2 = \frac{\Delta^2}{12}$ (for uniform distribution).
  - **The 6 dB per Bit Approximation**
    - Adding 1 bit doubles the number of quantization levels → halves the step size
    - This reduces noise power by 1/4, improving SNR by approximately **6.02 dB**.
      - $SNR_{dB} = 10log_{10}(\frac{P_{signal}}{P_{noise}})$

$$\Delta_{new} = \frac{\Delta_{old}}{2}$$

→ $\Delta\text{SNR}(dB) = 10log_{10}(\frac{P_{noise_old}}{P_{noise_new}}) \rightarrow 10log_{10}\left(\frac{1}{1/4}\right) = 10log4 \approx 6.02\ dB$

For each additional bit of resolution, the SNR improves by roughly 6.02dB

Quantization Noise Model

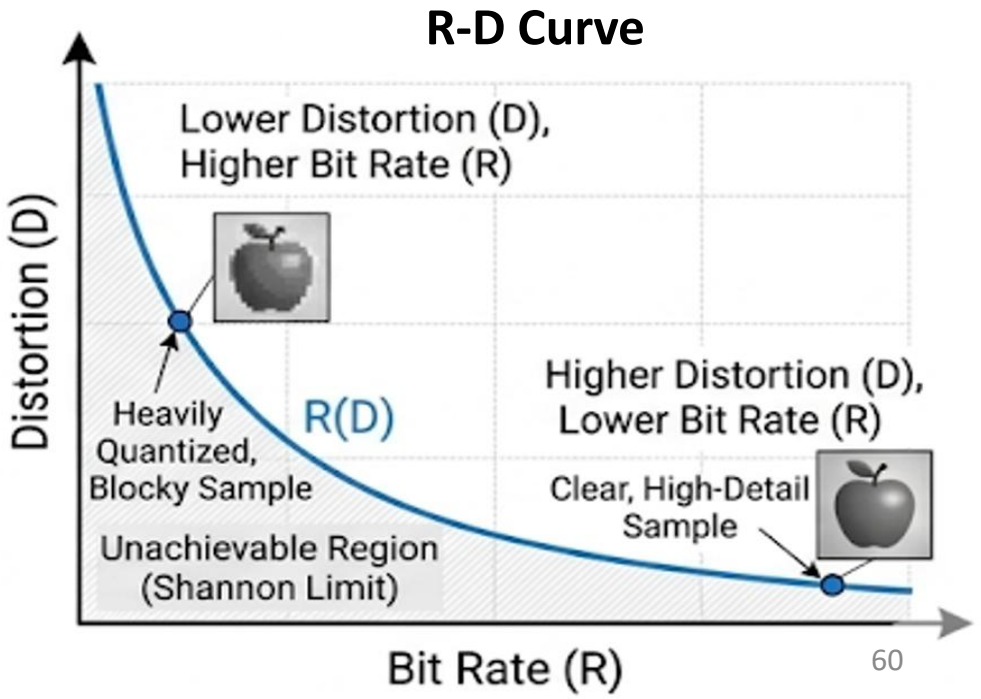
- We can use the **6dB rule** to roughly estimate the quantization-limited Dynamic Range of an ideal digital systems

  Audio Example

  - **8-bit Systems** (Early Game Consoles)
    - 8 bits × 6dB/bit ≈ 48 dB
    - Quantization noise may become noticeable, especially at low signal levels.
  - **16-bit Systems** (Standard CD Quality)
    - 16 bits × 6dB/bit ≈ 96 dB
    - Quantization noise is below the audible threshold under normal listening conditions.
  - **24-bit Systems** (Professional Studio Recording)
    - 24 bits × 6dB/bit ≈ 144 dB
    - An enormous dynamic range that far exceeds both human hearing limits and the noise floor of most hardware

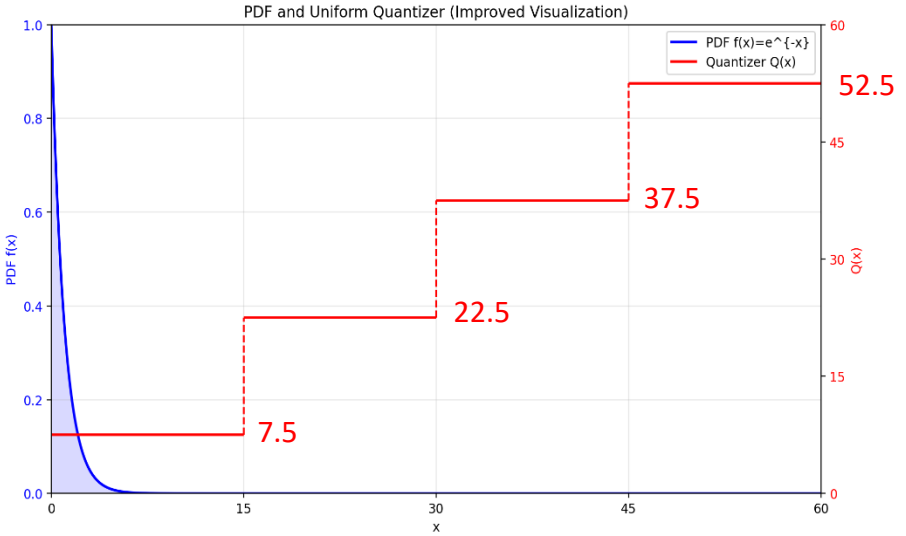
Rate–Distortion Theory

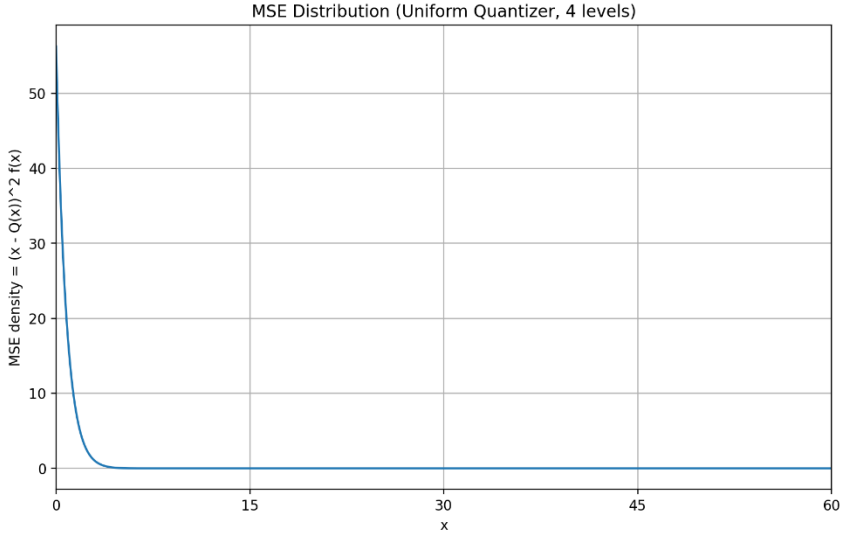
- **Balancing Efficiency: Bit Rate vs. Distortion**
  - **Bit Rate (R):** The average number of bits required to represent the source. Often expressed as **bits/sample**, **bits/pixel**, or **bits/second (bps)**, depending on application.
  - **Distortion (D):** The difference between the original and reconstructed signal. Common measures include **MSE**, **PSNR**, or perceptual distortion metrics
  - **R-D Optimization**: Minimize D for a given R, or minimize R for a given D constraint.
  - **R-D Curve**
    Describes the best theoretically achievable trade-off between bitrate and distortion for a given source. Lower distortion generally requires a higher rate.
  - **Shannon's Rate–Distortion Theorem**
    Any point below the curve is physically impossible to achieve.



Non-Uniform Quantization

- **Why Use Non-Uniform Quantization?**
  - **Uniform Limitation**: Treats all input x equally likely.
  - Real-world signals (speech, images) are not uniform; small amplitudes occur more frequently.
  - **Strategy**: Use high-density (smaller) steps where the probability density function (PDF) is high, and sparse (larger) steps where the PDF is low.
  - **Goal**: Minimize the total Mean Squared Quantization Error (MSQE)
- **Lloyd-Max Quantization**
  - Optimizing quantization for signal distribution





Lloyd–Max Quantization Algorithm

- **Problem Formulation**
  - Given a random variable X with probability density function $f(x)$, design an M-level scalar quantizer that minimizes the mean squared error (MSE)

$$D = E[(x - Q(x))^2] = \sum_{i=1}^{M} \int_{b_{i-1}}^{b_i} (x - y_i)^2 f_X(x)\, dx$$

  - where:
  - b_i : decision boundaries
  - y_i : reconstruction levels
  - $f_X(x)$: the input signal probability density function
  - $\mathrm{Q}(x) = y_i$ for $x \in [b_{i-1}, b_i)$

Lloyd–Max Quantization Algorithm

- **Optimality Conditions**
  - The Lloyd–Max algorithm is based on two necessary conditions for optimality
  - **(a) Boundary Condition**
    - Each decision boundary lies at the midpoint between adjacent reconstruction levels:

$$b_i = \frac{y_i + y_{i+1}}{2}, \quad i = 1, 2, \ldots, M-1$$

  - **(b) Centroid Condition**
    - Each reconstruction level is the conditional expectation (centroid) of its region:

$$y_i = \frac{\int_{b_{i-1}}^{b_i} x f_X(x)\, dx}{\int_{b_{i-1}}^{b_i} f_X(x)\, dx}, \quad i = 1, 2, \ldots, M$$

Lloyd–Max Quantization Algorithm

Algorithm Procedure

- Given the number of levels M
- **Step 0:** Choose initial levels

$$y_1^{(0)}, y_2^{(0)}, \ldots, y_M^{(0)}$$

  - e.g., Uniformly spaced levels, or data-driven initialization

- **Step 1:** Update Boundaries

$$b_i^{(t)} = \frac{y_i^{(t)} + y_{i+1}^{(t)}}{2}, \quad i = 1, \ldots, M-1$$

  - Set b_0, b_M according to the support of X
  - (e.g., $b_0 = 0, b_M = \infty$)

- **Step2:** Update Levels

$$y_i^{(t+1)} = \frac{\int_{b_{i-1}^{(t)}}^{b_i^{(t)}} x f_X(x)\, dx}{\int_{b_{i-1}^{(t)}}^{b_i^{(t)}} f_X(x)\, dx}$$

- **Step 3:** Convergence Check
  - Stop if, for all i, distortion change is below a threshold

$$|y_i^{(t+1)} - y_i^{(t)}| < \epsilon$$

  - Otherwise, return to Step1.

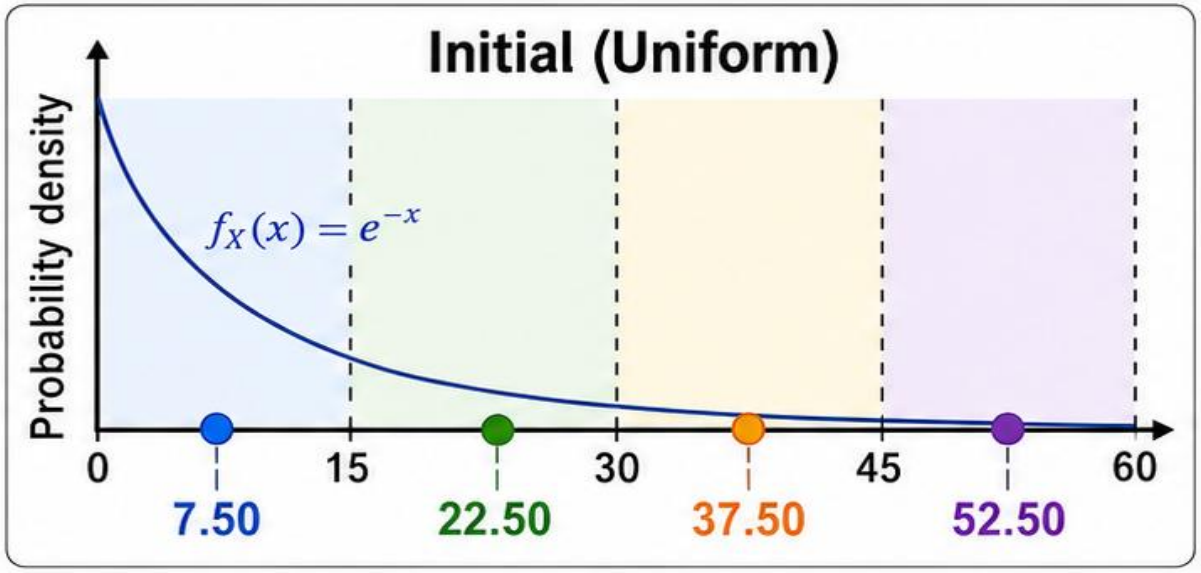
Lloyd–Max Example

- **Input:** $f_X(x) = e^{-x},\ 0 \le x \le 60$

  4-level quantizer: $M = 4$

  Initial levels: $y = [7.5, 22.5, 37.5, 52.5]$

- **Step1: update boundaries**
  - $b_1 = \frac{7.5+22.5}{2} = 15,\ \ b_2 = \frac{22.5+37.5}{2} = 35, b_3 = \frac{37.5+52.5}{2} = 45$



Input:
- pdf $f_X(x)$
- number of levels M
- initial levels y_i, …, y_M
- tolerance ε

Repeat:

1. Update boundaries:

$$b_i = (y_i + y_{i+1}) / 2$$

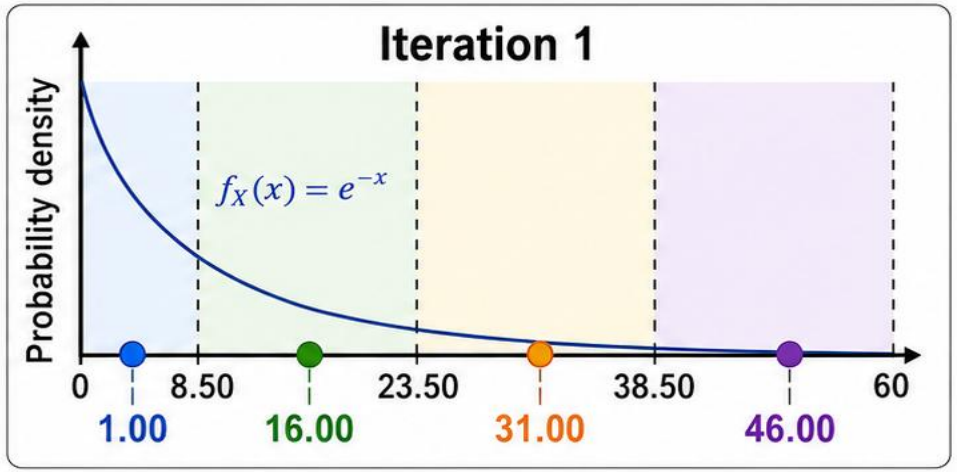
2. Update levels:

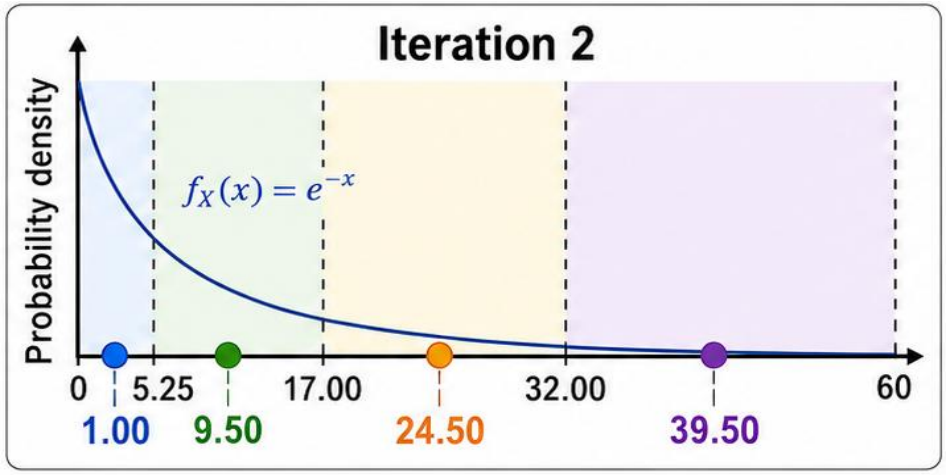
$$y_i = \frac{\int_{b_{i-1}}^{b_i} x f_X(x) dx}{\int_{b_{i-1}}^{b_i} f_X(x) dx}$$

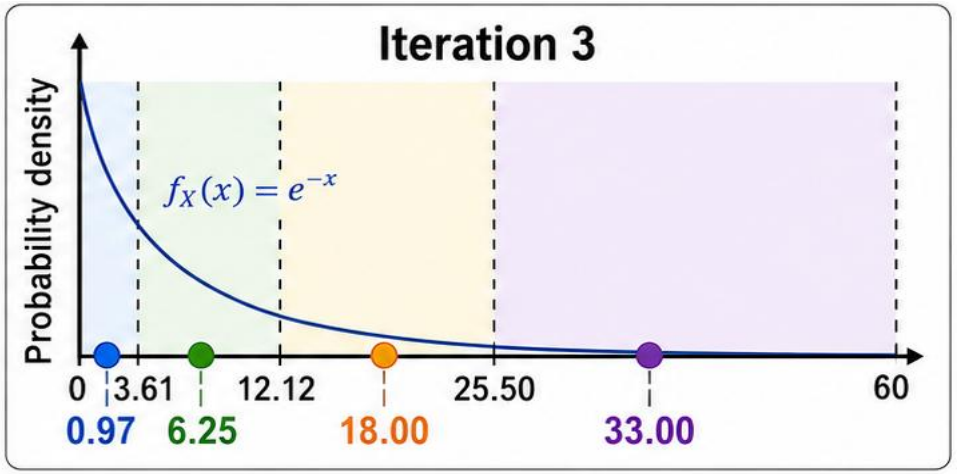
3. Check convergence

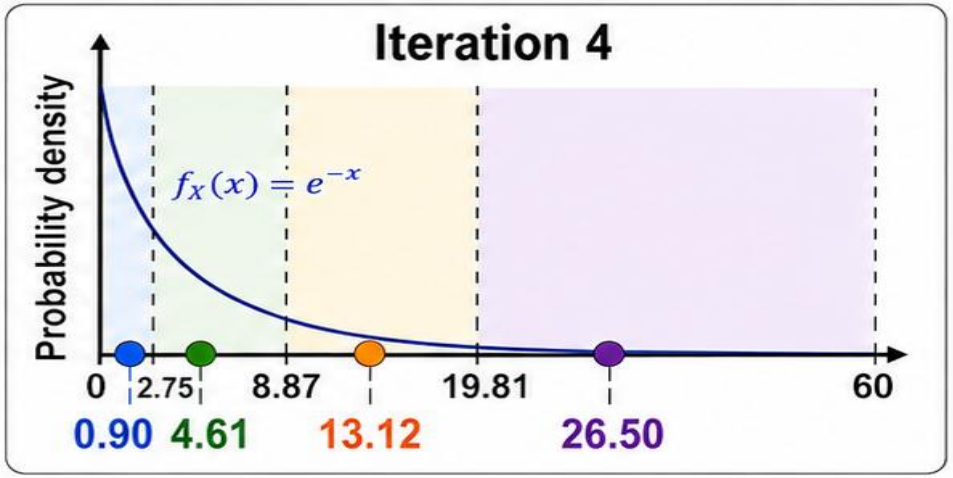
Until convergence

Lloyd–Max Example





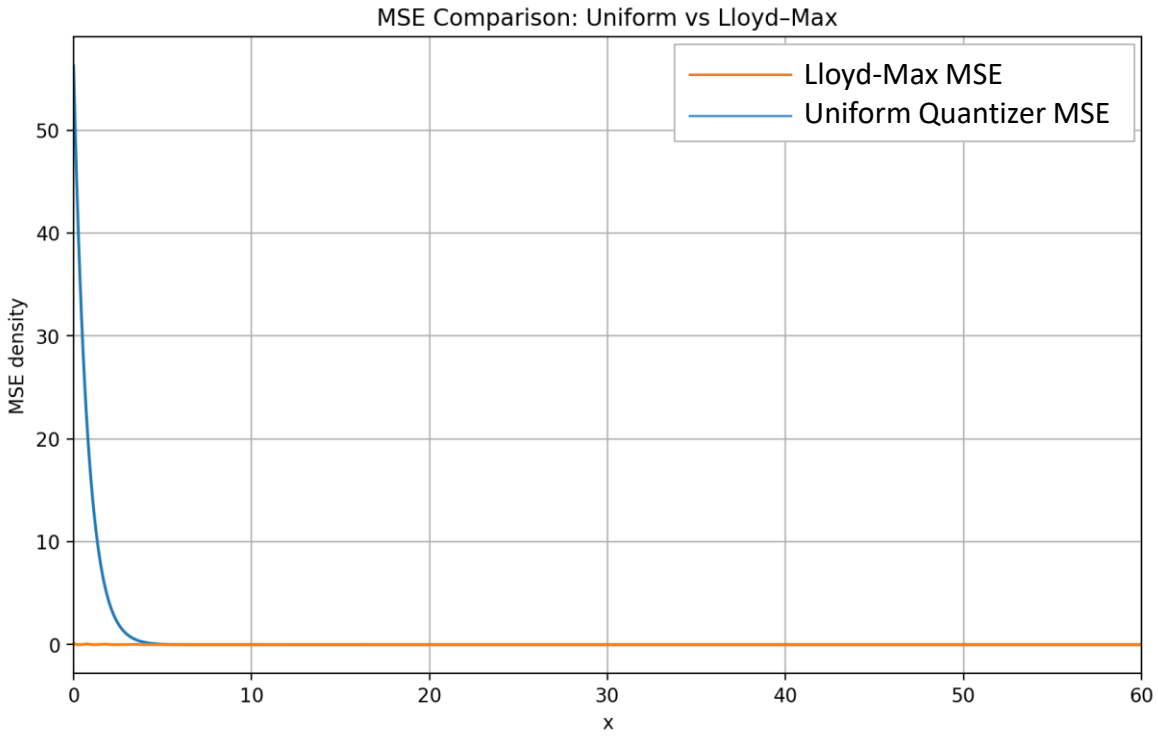




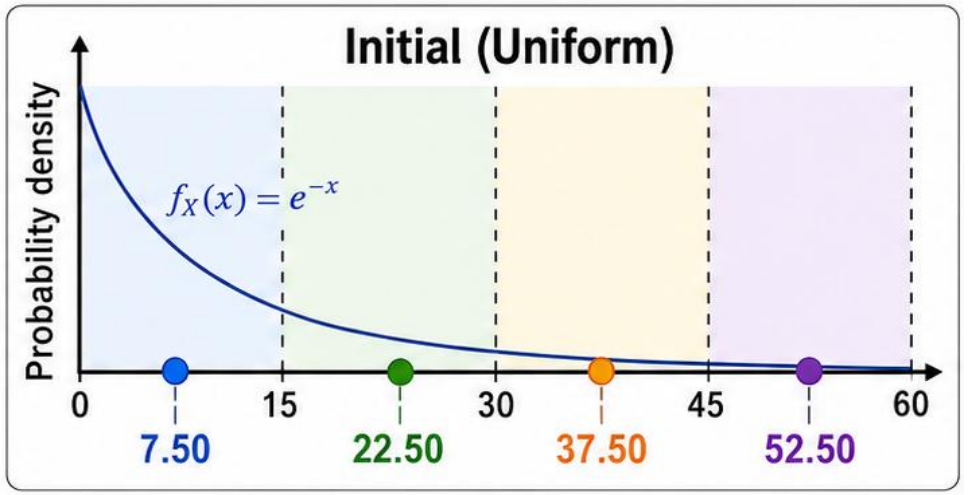
Because the source probability is concentrated near x=0, Lioyd-Max moves decision boundaries and reconstruction levels toward the high-probability region, producing non-uniform quantization.

MSE Comparison

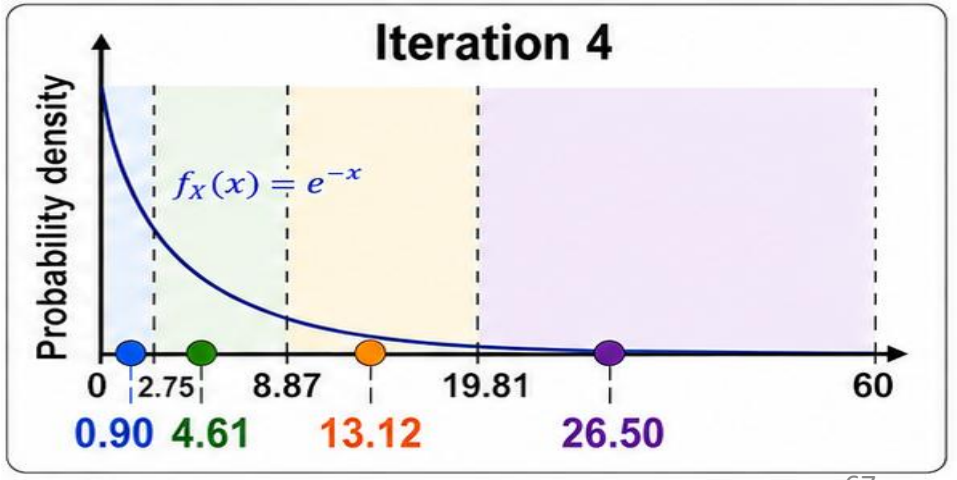
- **Lloyd-Max vs Uniform Quantizer**



Uniform Quantizer



Lloyd-Max Quantizer



Vector Quantization (VQ)

- **From Scalar to Vector Quantization**
  - **Scalar Quantization (SQ):** Processes each sample (e.g., a single pixel) independently. This is analogous to placing points on a Square Grid, which ignores the spatial relationships between neighbors.
  - **Vector Quantization (VQ):**
    - Groups a set of samples (e.g., 4x4 blocks) into a single k-dimensional vector and finds the best representative point in k-dimensional space.
    - **The VQ Gain**

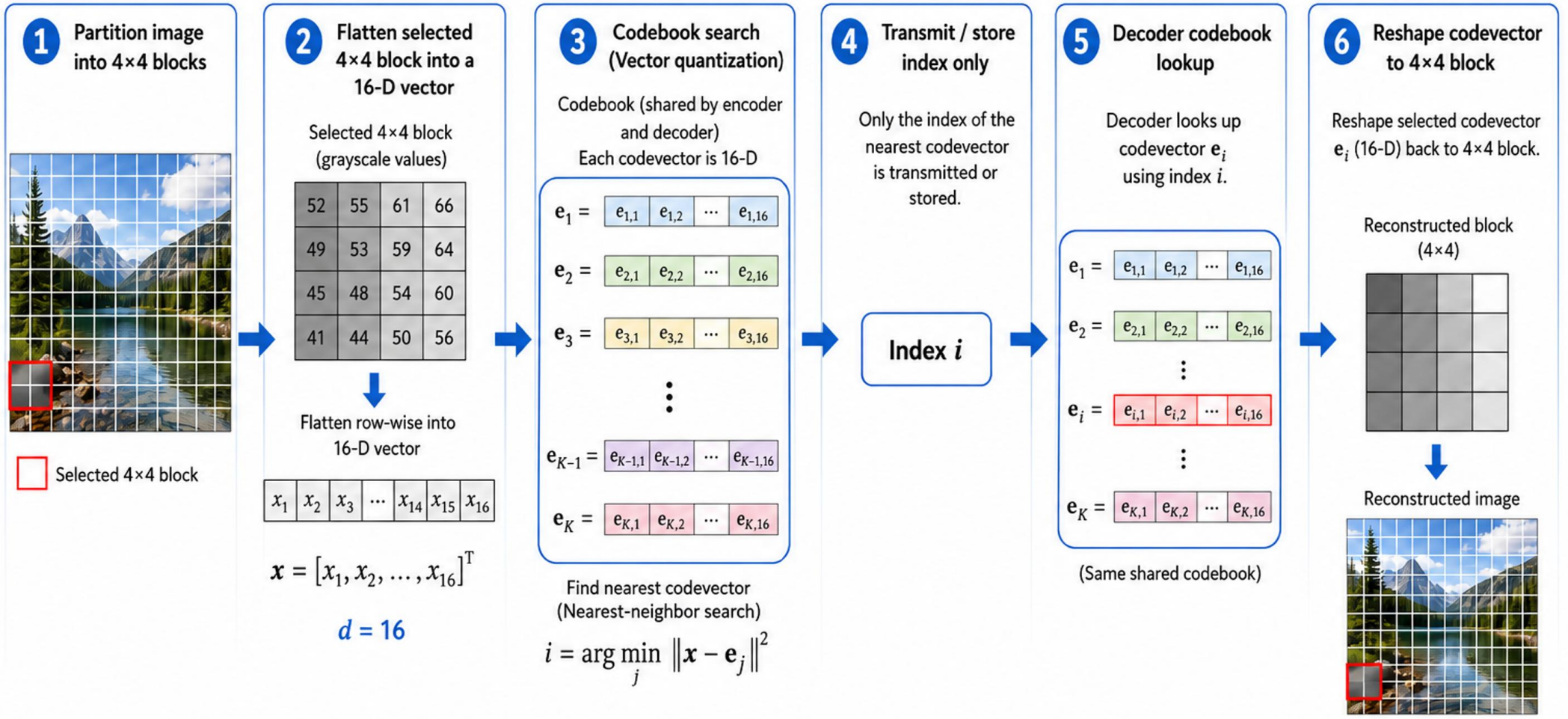
      Exploiting Correlation: Since adjacent pixels are often highly similar, VQ captures this structure more efficiently than SQ.

Vector Quantization (VQ)

- **VQ (Vector Quantization):** Encoding via Codebook Search
- **Codebook:** A predefined set of representative vectors called Codevectors.
- **Encoding Process**
  - Partition the input image into blocks (e.g., 4x4)
  - flatten each block into a high-dimensional vector (e.g., 16-D vectors)
  - Search the codebook for the **Codevector** that minimizes distortion (e.g., MSE)
  - Transmit only the **Index** of that codevector.
- **Decoding Process**
  - Receive the index, and perform table Look-up in the codebook to retrieve the corresponding codevector

VQ decoding is very fast, while Encoding is computationally expensive due to the high-dimensional search.

Vector Quantization (VQ)



- **Each 4x4 image block is represented by the index of its nearest codevector.**
- **Repeat for all image blocks.**

Design the Codebook

- **The LBG Algorithm (Linde-Buzo-Gray)**
  - A classic algorithm for vector-quantizer codebook design**.**
  - Designing an optimal codebook is fundamentally a Data Clustering problem
  - It is closely related to **K-means clustering**, where each codevector acts as a cluster centroid
- **Iterative Optimization**
  - **Partitioning / Assignment**: Assign each training vector to the nearest codevector according to a distortion measure (e.g., Euclidean distance or MSE).
  - **Centroid Update**: Calculate the **Centroid** (mean) of all data points within each region and set it as the new codevector.
  - **Check Convergence**: Repeat until the total distortion stabilizes.

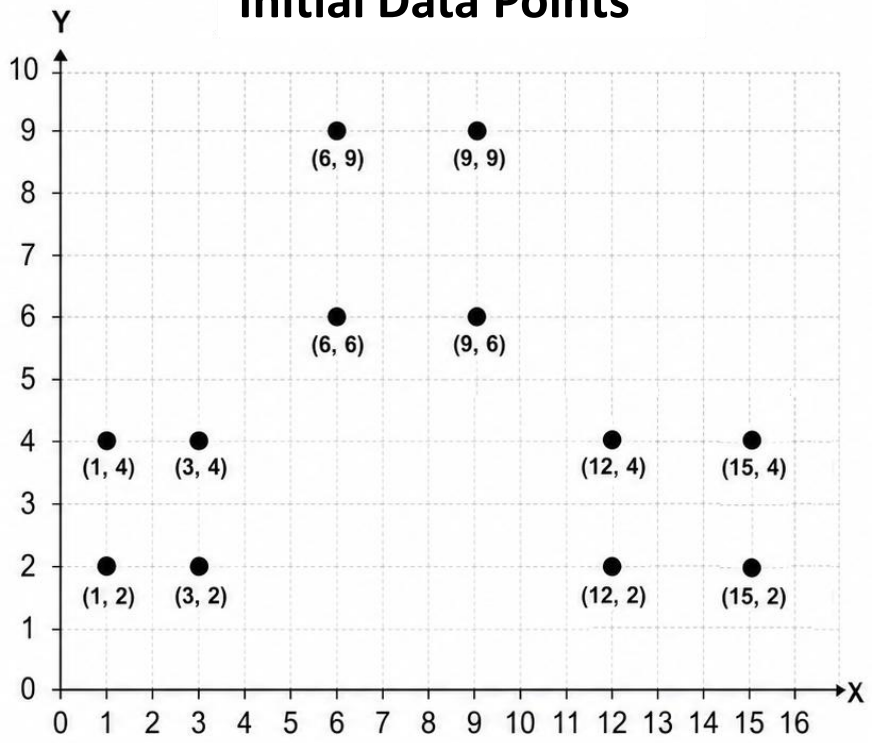
LBG alternates between assignment and centroid updates to reduce quantization distortion.

K-means Algorithm

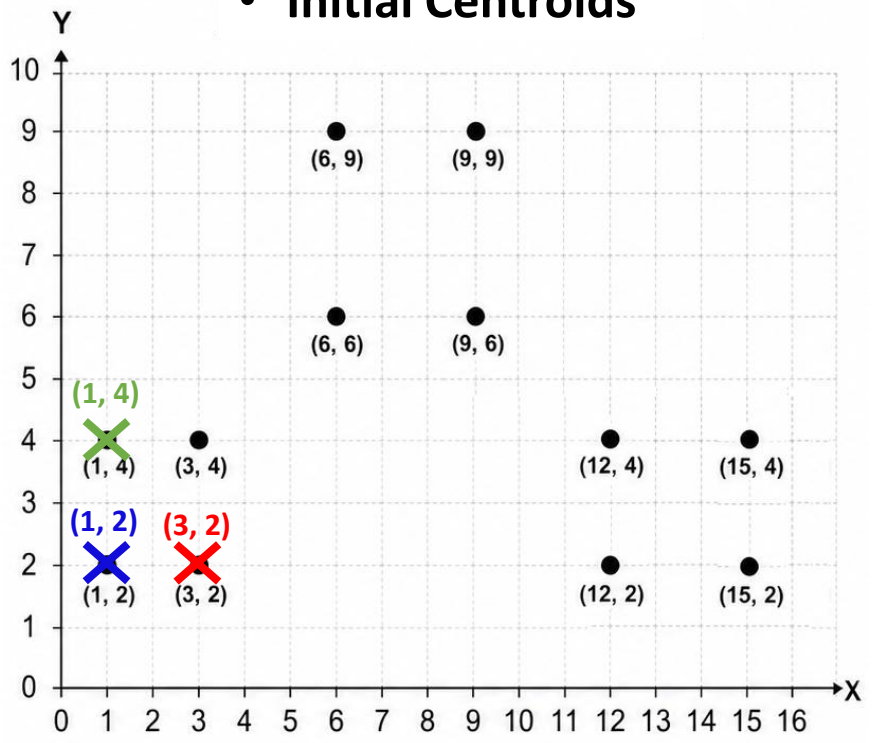
- Scenario: We have 12 data points in 2D space (e.g., use 1x2 pixel blocks)
- Objective: Design a Codebook with three vectors (k=3)
- Initial Centroid: randomly selected

Initial Stage

- Initial Data Points



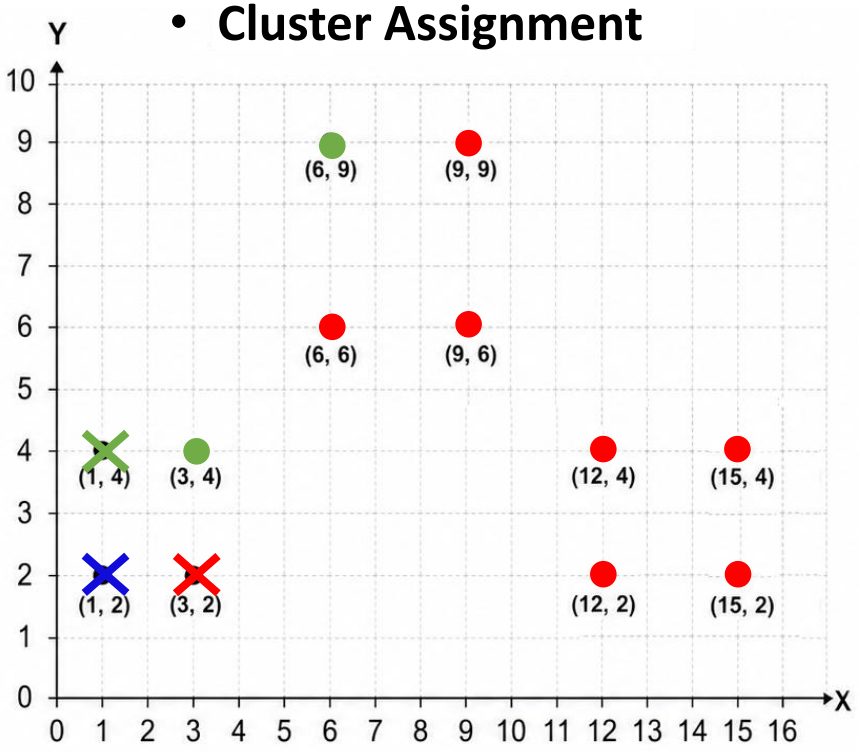
- Initial Centroids



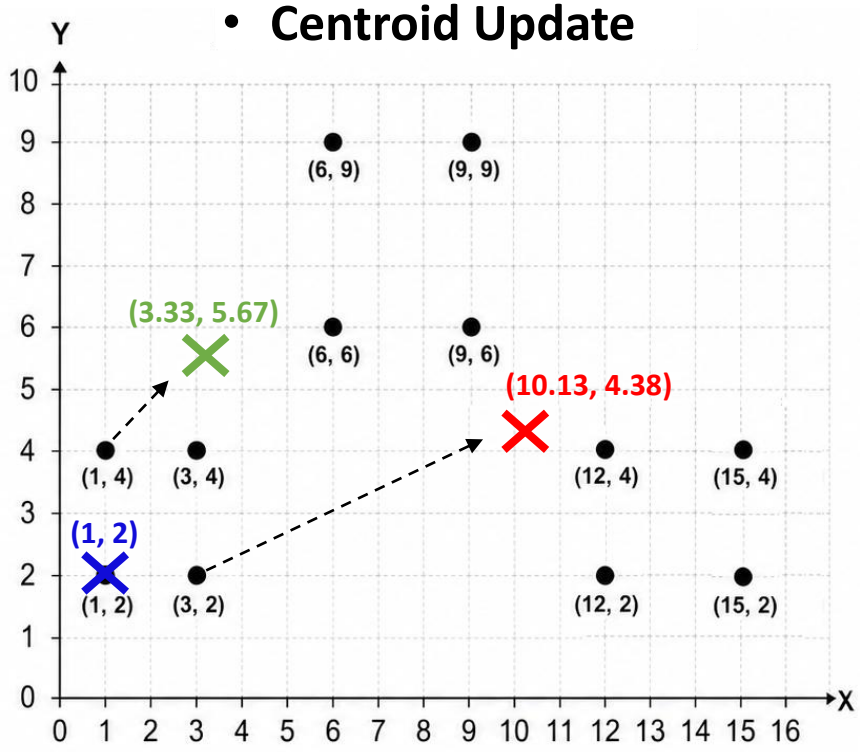
K-means Algorithm

Iteration 1

- Cluster Assignment



- Centroid Update



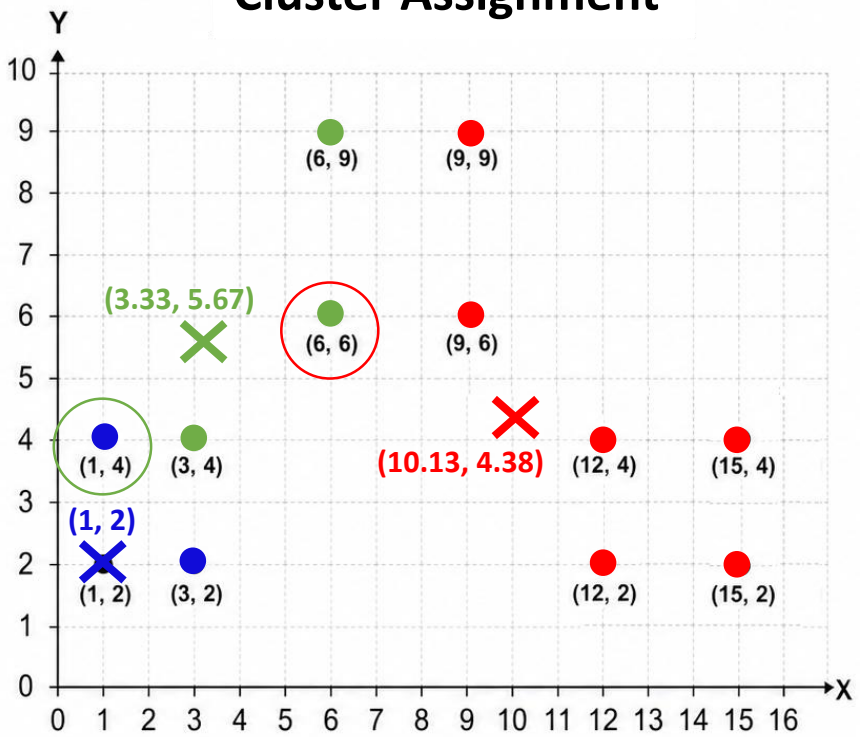
$$\mu_2^{(1)} = \left(\frac{1+3+6}{3}, \frac{4+4+9}{3}\right) = \boxed{(3.333,\ 5.667)}$$

$$\mu_3^{(1)} = \left(\frac{3+6+9+9+12+12+15+15}{8}, \frac{2+6+6+9+2+4+2+4}{8}\right) = \boxed{(10.125,\ 4.375)}$$

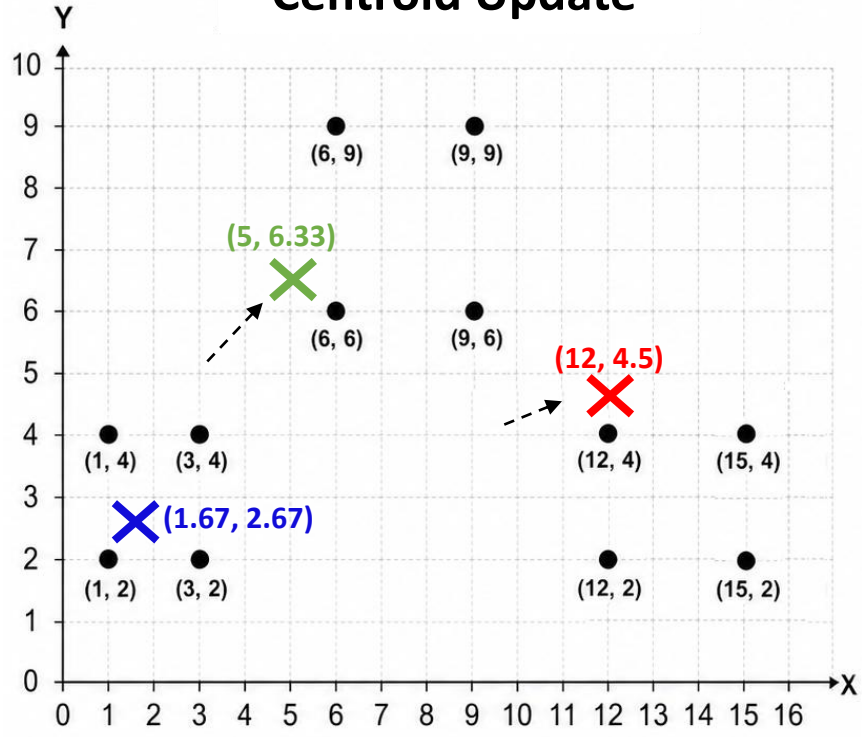
K-means Algorithm

Iteration 2

- **Cluster Assignment**



- **Centroid Update**



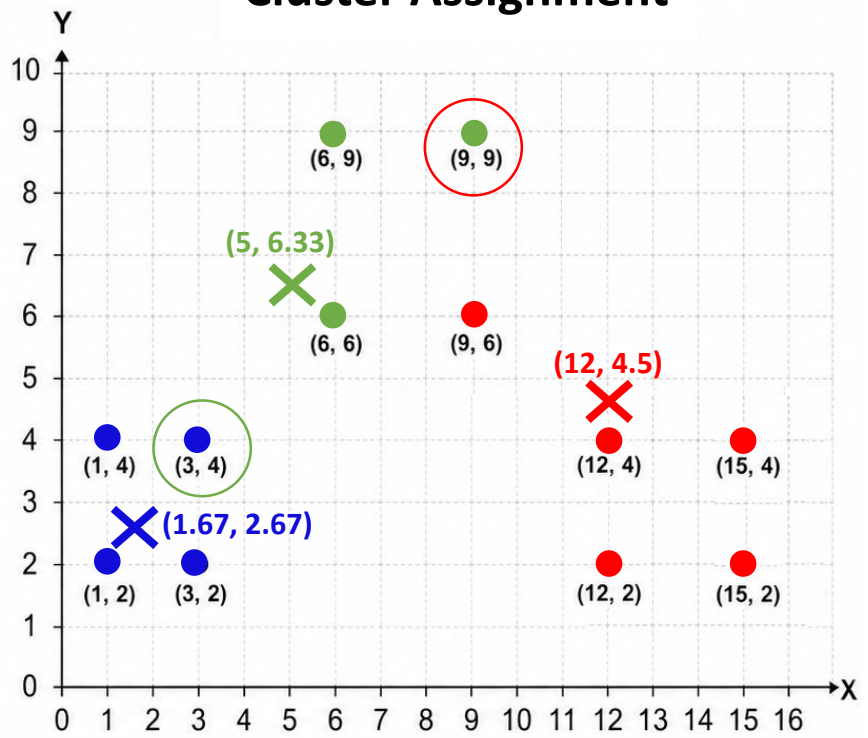
$$\times \quad \mu_1^{(2)} = \left(\frac{1+1+3}{3}, \frac{2+4+2}{3}\right) = \boxed{(1.667,\ 2.667)}$$

$$\times \quad \mu_2^{(2)} = \left(\frac{3+6+6}{3}, \frac{4+6+9}{3}\right) = \boxed{(5,\ 6.333)}$$

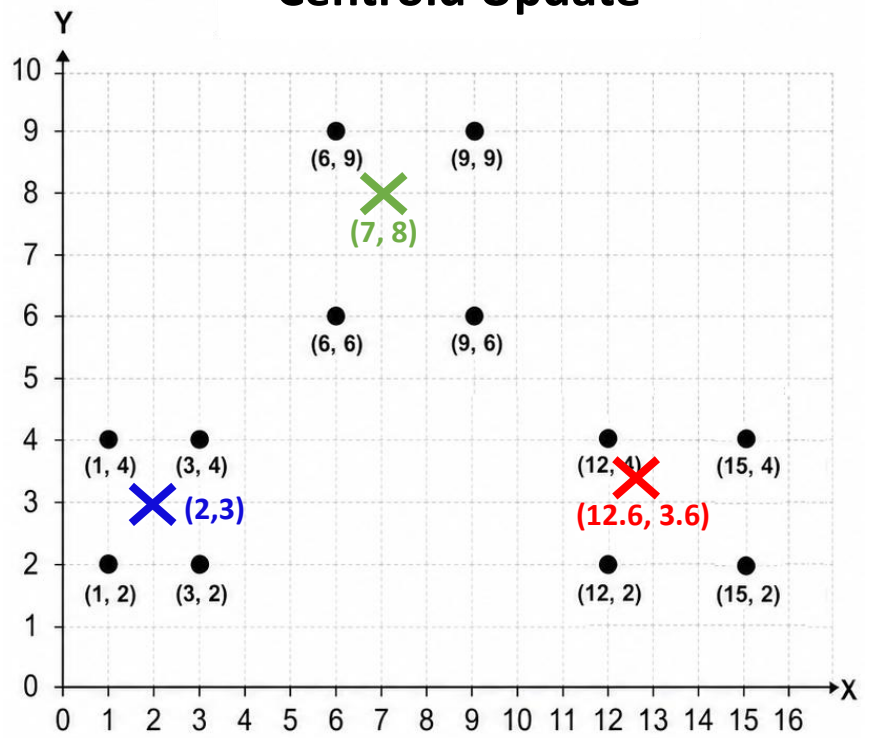
K-means Algorithm

Iteration 3

- **Cluster Assignment**



- **Centroid Update**



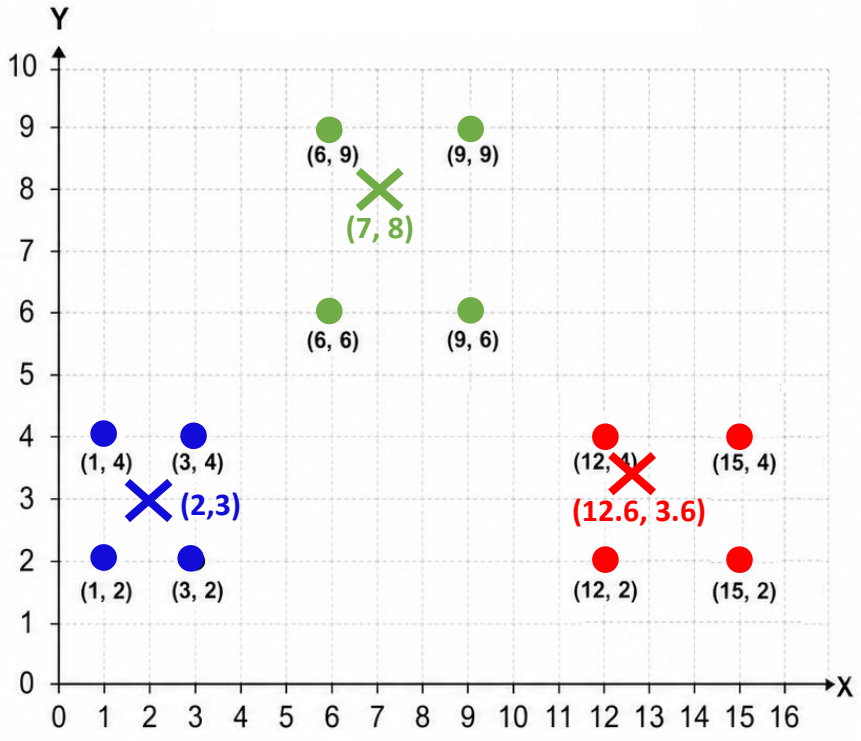
$$\mu_2^{(3)} = \left(\frac{6+6+9}{3}, \frac{6+9+9}{3}\right) = \boxed{(7,8)}$$

$$\mu_3^{(3)} = \left(\frac{9+12+12+15+15}{5}, \frac{6+2+4+2+4}{5}\right) = \boxed{(12.6, 3.6)}$$

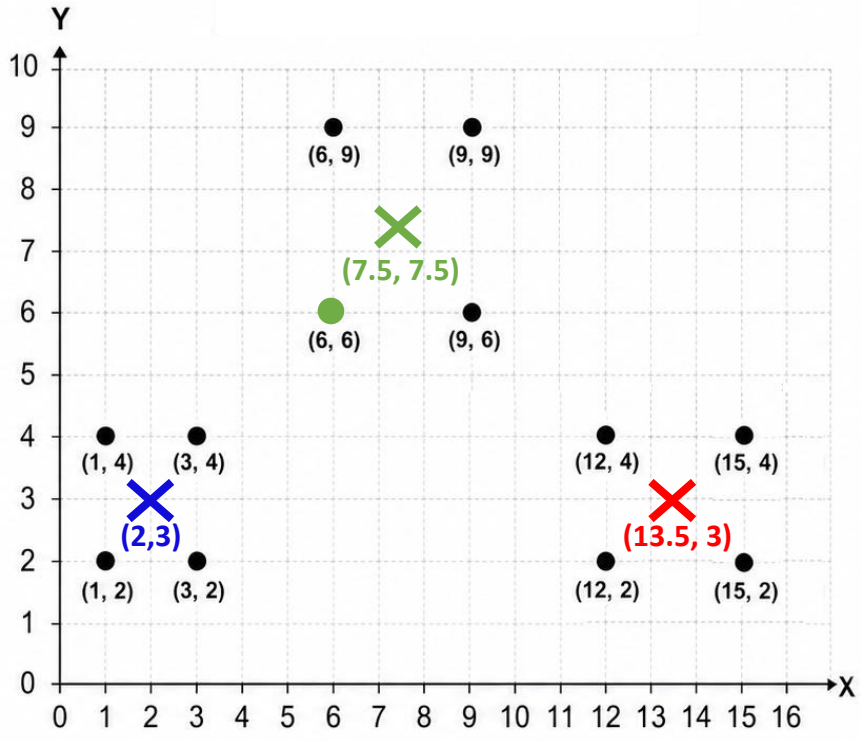
K-means Algorithm

Iteration 4

- **Cluster Assignment**



- **Centroid Update**



Iteration 5

- **Cluster Assignment**

No change

Converged

From VQ to VQ-VAE

- **Traditional VQ**
  - Operates on: raw pixel data (or fixed, hand-designed representations).
    - **Input Vector**: A 4x4 grayscale block can be flattened into a **16-dimensional** vector

      $$x = [p_1, p_2, \ldots, p_{16}]$$

    - **Quantization:** Search the codebook for the nearest codevector

      $$i^* = arg\ min_i \|x - c_i\|^2 \qquad \text{for all } i \text{ in the Codebook}$$

  - **Key Limitation**: **"curse" of Dimension**
    - As the block size increases, the vector dimension grows rapidly.

      4x4 blocks → dim = 16 16x16 blocks → dim = 256

    - A higher-dimensional space requires a much larger codebook to represent diverse input patterns well. → increases codebook storage and search complexity.
    - The high-dimension representation itself is not **learned jointly** with the quantizer.

→ VQ-VAE learns a compact latent representation before vector quantization.

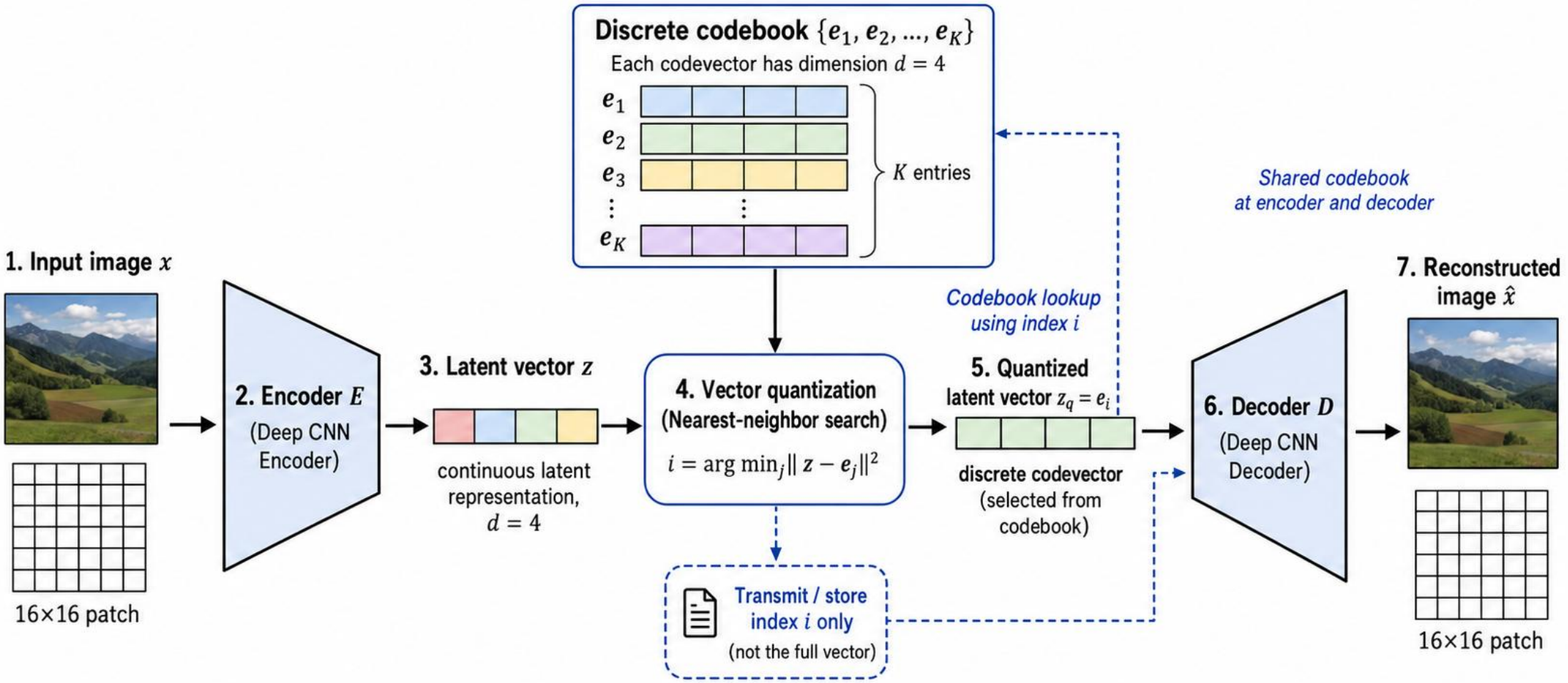
VQ-VAE (Variational AutoEncoder)

- **VQ-VAE (AI Codec)**
  - Operates on Latent Space (a compressed representation learned by a neural network) instead of raw pixels.
  - A neural encoder transforms the input into a lower-dimensional latent vector
  - Vector quantization is then performed in the **latent space**:
    - z = E(x) → $i^* = arg\ min_i \|z - e_i\|^2$ → $z_q = e_{i^*}$
  - A neural decoder reconstructs the signal: $\hat{x} = D(z_q)$

X → Encoder → z → Nearest Codevector → z_q → Decoder → $\hat{x}$

- **Why is this better**
  - The encoder learns to remove irrelevant details and quantization is performed in a more compact and **quantization-friendly latent space**..
  - The codebook represents learned features rather than raw pixel patterns.
  - Encoder, codebook, and decoder are trained jointly toward good reconstruction.

Vector Quantization with a Discrete Codebook



Nearest-codevector assignment converts a continuous latent into a discrete representation

Traditional VQ vs. VQ-VAE

Feature	Traditional VQ	VQ-VAE (Deep VQ)
Data Domain	Raw (pixel)/ fixed $x \rightarrow$ VQ	Learned latent $x \rightarrow$ E(x) $\rightarrow$ VQ
Codebook Design	LBG / K-means	Learned jointly with encoder/decoder
Search Complexity	**Very High** (High-dimensional codevector space)	**Moderate** (Low-dimensional latent space search)
Compression Efficiency	**Moderate** (Good for textures, poor for edges)	**High** (Learns complex, non-linear structures)
Current Status	Obsolete in Mainstream Video	The Core of Modern AI Compression (e.g., DALL-E)

Why VQ is "Missing" from Modern Video Standards

- **Practical Trade-offs: VQ vs. Transform Coding**
  - **The Complexity Bottleneck of VQ**:
    - As vector dimensions increase, the time required to search the codebook grows exponentially.
    - Real-time 4K/8K video encoding requires speeds that traditional VQ simply cannot meet with standard hardware.
  - **The Rise of Transform Coding**
    - Mainstream coding standards use **Transforms** to decorrelate data, followed by simple **Scalar Quantization**. This achieves compression ratios close to VQ but with vastly superior computational efficiency.
  - **Modern Rebirth: AI-based Compression**
    - In Deep Learning-based codecs, VQ is seeing a massive revival through **VQ-VAE**, where the latent space is quantized using VQ principles.